

SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing

BSc (Hons) in Information Technology

IT4030 – Internet of Things

GreenPulse

A HEALTHIER HOME, A HAPPIER PLANT

Smart Agentic IoT Plant-Care Device with LLM Backend

PROJECT PROPOSAL

Submitted By:

Student Name	Student ID
Kumarathunga R C S	IT23257054
Wathsala K L K	IT23182882
Senanayake Y M	IT23284470
Methsuka S A I	IT23160484

Submitted To: Mr. Uditha Dharmakeerthi.

Academic Year: 2026

Submission Date: 20 September 2026

Table of Contents

CHAPTER 1: INTRODUCTION	7
1.1 Background of the Study	7
1.2 Problem Statement	8
1.3 Proposed Solution	9
1.4 Project Aim.....	9
1.5 Specific Objectives.....	10
1.6 Project Scope.....	10
1.6.1 Hardware Scope.....	10
1.6.2 Communication Scope	10
1.6.3 Cloud and Intelligent Processing Scope.....	11
1.6.4 Dashboard Scope	11
1.6.5 Project Limitations	11
CHAPTER 2: PROPOSED SYSTEM DESIGN	12
2.1 System Overview.....	12
2.2 Overall System Architecture.....	14
2.2.1 Sensing Layer.....	14
2.2.2 Device Processing Layer	14
2.2.3 Communication Layer	14
2.2.4 Cloud Intelligence Layer	14
2.2.5 Visualization Layer.....	14
2.3 Hardware Components and Sensor Selection.....	15
2.3.1 ESP32 Microcontroller	16
2.3.2 Soil Moisture Sensor	16
2.3.3 DHT22 Temperature and Humidity Sensor	16
2.3.4 BH1750 Light Intensity Sensor.....	16
2.3.5 OLED Display	16
2.3.6 RGB LED Priority Indicator	17
2.4 Proposed Circuit Design and GPIO Configuration.....	17
2.5 ESP32 Firmware Design.....	18
2.5.1 System Initialization.....	18
2.5.2 Sensor Data Acquisition	18
2.5.3 Local Display and Priority Processing.....	19
2.5.4 MQTT Data Publication	19
2.5.5 MQTT Subscription and Connection Management	19
2.6 OLED Display and Priority Indicator Design	20

2.6.1 OLED Display Layout	20
2.6.2 Priority Indicator Logic	20
2.6.3 Local Interface Design Considerations	21
CHAPTER 3: COMMUNICATION AND CLOUD ARCHITECTURE.....	22
3.1 Secure MQTT Communication	22
3.1.1 Wi-Fi Connectivity	22
3.1.2 MQTT Broker Selection	22
3.1.3 Transport Layer Security	23
3.1.4 Communication Reliability	23
3.2 MQTT Topics and JSON Payloads	24
3.2.1 Sensor Telemetry Payload.....	24
3.2.2 Device Status Payload	25
3.2.3 AI-Generated Care Message Payload	25
3.2.4 Weather and Notification Payloads	25
3.2.5 Payload Validation	26
3.3 Cloud Backend Architecture.....	26
3.3.1 MQTT Subscriber Service.....	26
3.3.2 Sensor Data Processing	26
3.3.3 External Service Integration	26
3.3.4 LLM Processing Service	27
3.3.5 MQTT Response Publisher	28
3.4 Data Storage Design	28
3.4.1 Sensor Data Collection	28
3.4.2 Proposed Database Structure.....	29
3.4.3 Data Storage and Retrieval	29
3.4.4 Data Management Considerations.....	29
3.5 Bidirectional Communication.....	29
3.5.1 Device-to-Cloud Communication	30
3.5.2 Cloud-to-Device Communication	30
3.5.3 End-to-End Communication Sequence	30
CHAPTER 4: INTELLIGENT SYSTEM DESIGN	31
4.1 LLM-Based Plant-Care Intelligence.....	31
4.1.1 LLM Framework and Processing Architecture	31
4.1.2 Sensor Data Preparation.....	31
4.1.3 Prompt Engineering.....	32

4.1.4 Plant-Care Tip Generation	32
4.1.5 Literary Quote Generation	33
4.1.6 LLM Output Validation and Publication	33
4.2 Real-Time Weather API Integration.....	34
4.2.1 Weather Data Source	34
4.2.2 Weather Data Collection	34
4.2.3 Weather-Aware LLM Processing	34
4.2.4 Weather Summary Publication	35
4.3 Email and Notification Integration	35
4.3.1 Email Service Selection	35
4.3.2 Notification Retrieval	35
4.3.3 LLM-Based Notification Summarization	36
4.3.4 Privacy and Security	36
4.3.5 Notification Summary Publication.....	36
4.4 Node-RED Dashboard Design.....	37
4.4.1 Live Sensor Monitoring	37
4.4.2 MQTT Communication Monitoring	37
4.4.3 LLM Input and Output Visualization	37
4.4.4 Proposed Dashboard Layout	37
4.5 Proposed Advanced Features.....	38
4.5.1 Additional Environmental Sensors	38
4.5.2 Data-Driven Watering Predictions.....	39
4.5.3 Enhanced Cloud Automation	39
4.5.4 Future System Expansion.....	39
CHAPTER 5: IMPLEMENTATION METHODOLOGY	40
5.1 Development Methodology.....	40
5.1.1 Phase 1: Hardware Preparation and Circuit Development.....	40
5.1.2 Phase 2: ESP32 Firmware Development.....	40
5.1.3 Phase 3: Secure MQTT Communication	41
5.1.4 Phase 4: Cloud Backend and LLM Development	41
5.1.5 Phase 5: Node-RED Dashboard Development	41
5.1.6 Phase 6: System Integration and Evaluation	41
5.2 System Integration Strategy	42
5.3 Testing and Evaluation Plan	42
5.3.1 Sensor Evaluation.....	44

5.3.2 Communication Evaluation.....	44
5.3.3 Intelligent Processing Evaluation	44
5.3.4 Dashboard Evaluation	44
5.4 Proposed Project Timeline.....	44
5.5 Group Member Responsibilities	45
5.6 Budget and Feasibility.....	46
5.6.1 Technical Feasibility	46
5.6.2 Economic Feasibility	47
5.7 Risk Management and Mitigation.....	47
CHAPTER 6: EXPECTED OUTCOMES AND CONCLUSION	48
6.1 Expected Outcomes.....	48
6.1.1 Functional IoT Monitoring Device	48
6.1.2 Local Monitoring and Priority Indication	48
6.1.3 Secure Bidirectional MQTT Communication	48
6.1.4 Cloud-Based Data Collection and Storage	49
6.1.5 LLM-Generated Plant-Care Information	49
6.1.6 Real Weather and Notification Summarization.....	49
6.1.7 Centralized Node-RED Dashboard.....	49
6.1.8 Integrated End-to-End Prototype.....	50
6.2 Project Deliverables.....	50
6.3 Expected Project Contribution	51
6.4 Future Development Opportunities	51
6.5 Conclusion.....	52

List Of Tables

Table 1- Proposed Hardware Components and Their Functions	15
Table 2- : Preliminary ESP32 GPIO Configuration	18
Table 3- Proposed OLED Display Information	20
Table 4- Proposed RGB LED Priority States	20
Table 5 - Proposed MQTT Topics and Message Directions	24
Table 6 - Proposed Sensor Data Storage Schema	29
Table 7 - Proposed Node-RED Dashboard Components	37
Table 8 - Proposed System Testing and Evaluation Plan	43
Table 9 - Proposed Project Implementation Timeline	44
Table 10 - Proposed Group Member Responsibilities	45
Table 11 - Proposed Bill of Materials and Budget Structure	46
Table 12 - Proposed Project Risks and Mitigation Strategies	47
Table 13 - Proposed GreenPulse Project Deliverables	50

List Of Figures

Figure 1- Proposed Overall System Architecture of GreenPulse	13
Figure 2 - : Proposed GreenPulse Hardware Circuit Diagram	17
Figure 3- Proposed Firmware Operation and Data Acquisition Flow.....	19
Figure 4 - Proposed Secure MQTT Communication Architecture of GreenPulse	23
Figure 5 - Proposed Cloud Backend and Data Processing Architecture of GreenPulse.....	28
Figure 6 - Proposed LLM-Based Plant-Care Processing Workflow of GreenPulse	33
Figure 7 - Proposed External Information Retrieval and LLM Summarization Workflow	36
Figure 8 - Proposed Node-RED Dashboard Interface for GreenPulse	38
Figure 9 - Proposed Development Methodology of GreenPulse	42

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

The Internet of Things (IoT) has introduced new opportunities for developing intelligent systems capable of monitoring physical environments and supporting data-driven decision-making. IoT systems combine sensors, microcontrollers, communication networks, and software platforms to collect and process information from real-world environments. In agriculture and household plant maintenance, these technologies can be used to monitor environmental conditions that influence plant growth and provide users with timely information about plant-care requirements.

Traditional plant maintenance often depends on manual observation and fixed watering schedules. However, plant requirements can vary according to soil moisture, ambient temperature, humidity, light availability, and changing weather conditions. Without continuous monitoring, users may find it difficult to identify unfavorable conditions at the appropriate time. Excessive watering, insufficient watering, and unsuitable environmental conditions may consequently affect plant health.

IoT-based plant monitoring systems address some of these limitations by collecting environmental measurements through connected sensors. Nevertheless, raw sensor readings alone may not provide sufficient guidance for users who lack technical or agricultural knowledge. A system that measures soil moisture or temperature can indicate the current environmental condition, but additional processing is required to convert these measurements into understandable plant-care information.

Recent developments in Large Language Models (LLMs) provide opportunities to extend conventional monitoring systems with natural-language information processing. An LLM-based backend can receive structured sensor information, incorporate relevant external information, and generate contextual responses that are easier for users to understand. When combined with IoT technology, this approach can support a plant-care system that performs both environmental monitoring and intelligent information generation.

This project proposes **GreenPulse**, an IoT-based plant-care system that integrates an ESP32 microcontroller, environmental sensors, secure MQTT communication, a cloud-hosted LLM backend, and a Node-RED dashboard. The proposed system will monitor plant-related environmental conditions and transmit sensor measurements to a cloud platform for storage and further processing.

The cloud backend will use sensor information together with real weather data and relevant notifications from an authorized email account to generate plant-care messages, literary-style quotes, and contextual summaries. The resulting information will be published through MQTT and presented through a Node-RED dashboard. A local display and priority indicator will provide immediate information about sensor measurements, connectivity, and plant-care status.

Through this integration, GreenPulse aims to demonstrate how conventional IoT monitoring can be extended with cloud computing and LLM-based processing to create a more informative and interactive plant-care solution.

1.2 Problem Statement

Plant maintenance requires regular observation of environmental conditions and appropriate responses to changing plant-care requirements. However, many household plant owners and small-scale growers rely on manual inspection and routine watering schedules rather than continuous environmental monitoring.

This approach presents several limitations. First, environmental changes may occur between manual inspections, preventing users from identifying unfavorable conditions promptly. Second, watering decisions based only on fixed schedules may not reflect the actual moisture condition of the soil. Third, users may have difficulty interpreting environmental measurements and determining suitable care actions.

Although conventional IoT plant-monitoring devices can collect and display sensor readings, some systems provide limited contextual interpretation of the collected information. Users may therefore need to interpret multiple sensor measurements independently.

Plant-care decisions can also be influenced by external conditions, including current weather and relevant notifications. A monitoring system that considers only local sensor readings may not provide a complete picture of the information available to the user.

Consequently, there is a need for an integrated system that can monitor plant-related environmental conditions, securely transmit and store measurements, and transform the collected information into understandable, context-aware plant-care messages.

The central problem addressed by this project is the **lack of an integrated plant-monitoring solution that combines real-time environmental sensing, secure IoT communication, external information retrieval, and LLM-based interpretation within a unified system.**

GreenPulse is proposed to address this problem by connecting an ESP32-based monitoring device to a cloud-hosted intelligent backend and a Node-RED dashboard. The system will provide live environmental measurements and generate contextual information using sensor data, real weather information, and relevant email notifications.

1.3 Proposed Solution

GreenPulse will be developed as a smart IoT plant-care system consisting of a physical monitoring device, a secure communication layer, a cloud-hosted intelligent backend, and a visualization dashboard.

The physical device will use an ESP32 microcontroller to collect environmental information from connected sensors. The proposed sensor configuration includes a soil moisture sensor, a DHT22 temperature and humidity sensor, and a BH1750 ambient light sensor, together with any additional sensors retained from the original GreenPulse design.

The ESP32 will process sensor readings and display live measurements and MQTT connection status through an OLED display. An RGB LED will provide a visual indication of the plant-care priority based on predefined conditions. The local display will be restricted to live sensor readings and MQTT connection status, while AI-generated messages and MQTT payload information will be presented through the dashboard rather than on the device screen.

The ESP32 will establish a Wi-Fi connection and communicate with a selected MQTT broker using Transport Layer Security (TLS). Sensor measurements will be transmitted as structured JSON messages through defined MQTT topics. The proposed communication architecture will support bidirectional messaging between the IoT device and the cloud backend.

A cloud-hosted backend will subscribe to the relevant MQTT topics, receive sensor measurements, and store the collected information with timestamps. It will also retrieve current weather information from a real weather API and relevant messages from an authorized email or notification account.

The backend will use an LLM-based processing framework to analyze the available information and generate plant-condition-inspired literary quotes, care tips, weather summaries, and notification summaries. These outputs will be published to designated MQTT topics.

A Node-RED dashboard will subscribe to the relevant MQTT messages and display live sensor measurements, MQTT topic information, timestamps, connection status, LLM input and output payloads, and generated messages.

The proposed solution will therefore establish an end-to-end information flow from physical environmental sensing to secure cloud communication, intelligent processing, and user-oriented visualization.

1.4 Project Aim

The main aim of this project is to **design, develop, and evaluate an IoT-based intelligent plant-care system that integrates real-time environmental monitoring, secure MQTT communication, cloud-based data processing, and Large Language Model technology to provide contextual plant-care information through a unified dashboard.**

1.5 Specific Objectives

The following specific objectives are proposed to achieve the overall project aim:

1. To design and develop an ESP32-based monitoring device capable of collecting plant-related environmental measurements using a soil moisture sensor, a temperature and humidity sensor, a light intensity sensor, and any additional sensors included in the finalized hardware design.
2. To implement a local OLED display that presents live sensor measurements and MQTT connection status, together with a priority-based RGB LED indicator for communicating predefined plant-care conditions.
3. To establish secure, bidirectional communication between the ESP32 and a selected MQTT broker using TLS, defined MQTT topics, and structured JSON messages.
4. To develop and deploy a cloud-hosted backend capable of receiving MQTT sensor messages, processing incoming information, and storing timestamped sensor measurements.
5. To integrate an LLM-based processing framework that generates plant-condition-inspired literary quotes and contextual care tips using structured environmental sensor data.
6. To integrate a real weather API and an authorized email or notification account, enabling the backend to retrieve relevant information and generate weather and notification summaries.
7. To develop a Node-RED dashboard that visualizes live sensor measurements, MQTT connection status, topic names, timestamps, LLM input and output payloads, and generated plant-care information.
8. To investigate and implement feasible advanced features that extend the basic system functionality, such as the use of additional environmental sensors or data-driven watering predictions.
9. To evaluate the proposed system through planned hardware, communication, cloud-processing, AI-integration, dashboard, and end-to-end functional tests.

1.6 Project Scope

The scope of GreenPulse will cover the design, implementation, integration, and evaluation of an IoT-based plant-monitoring prototype with a cloud-hosted intelligent backend.

1.6.1 Hardware Scope

The hardware scope will include an ESP32 microcontroller, environmental sensors, an OLED display, an RGB LED indicator, and the supporting electronic components required to construct the prototype.

The device will collect soil moisture, ambient temperature, humidity, and light intensity measurements, along with readings from any additional sensors included in the finalized design.

The proposed prototype will provide local sensor visualization and priority indication.

1.6.2 Communication Scope

The communication scope will include Wi-Fi connectivity and secure MQTT communication using TLS.

The ESP32 will publish environmental measurements to predefined MQTT topics and subscribe to relevant cloud-generated messages. The system will define structured JSON payload formats and implement connection-management procedures.

1.6.3 Cloud and Intelligent Processing Scope

The cloud scope will include a deployed backend service that receives sensor data, stores timestamped measurements, retrieves external information, and coordinates LLM-based processing.

The intelligent processing component will generate plant-care messages and literary-style quotes using sensor information. It will also summarize real weather information and relevant messages obtained from an authorized email or notification account.

1.6.4 Dashboard Scope

The Node-RED dashboard will provide a centralized interface for monitoring live environmental measurements and viewing the information exchanged through MQTT.

It will present connection status, topic names, timestamps, LLM input and output payloads, and AI-generated care and summary messages.

1.6.5 Project Limitations

GreenPulse will initially be developed as a prototype intended to demonstrate the integration of IoT monitoring, secure communication, cloud processing, and LLM-based information generation.

The accuracy of its environmental measurements will depend on the selected sensors, their calibration, and the conditions under which the prototype is tested.

Cloud functionality will depend on network connectivity and the availability of the selected MQTT broker, cloud services, weather API, email integration, and LLM service.

LLM-generated information will be treated as contextual assistance rather than guaranteed horticultural advice. The project will evaluate whether the generated outputs are relevant to the available input data.

Automatic watering or other physical actuation will not be assumed to be part of the core implementation unless it is explicitly included in the finalized hardware and advanced-feature design.

The initial evaluation will focus on the prototype's functional operation and end-to-end integration rather than claiming proven long-term improvements in plant growth or agricultural productivity.

CHAPTER 2: PROPOSED SYSTEM DESIGN

2.1 System Overview

GreenPulse is proposed as an intelligent Internet of Things (IoT) plant-care system that combines environmental sensing, embedded processing, secure cloud communication, and Large Language Model (LLM)-based information generation. The system will be designed to monitor environmental conditions associated with plant maintenance and provide users with real-time measurements and contextual plant-care information.

The proposed architecture will consist of four major subsystems: the IoT monitoring device, secure MQTT communication infrastructure, cloud-based intelligent processing backend, and Node-RED visualization dashboard.

The IoT monitoring device will be developed using an ESP32 microcontroller connected to environmental sensors. A soil moisture sensor will measure moisture conditions in the soil, a DHT22 sensor will collect ambient temperature and relative humidity measurements, and a BH1750 sensor will measure ambient light intensity. Additional sensors included in the finalized GreenPulse hardware design will also be integrated into the monitoring subsystem.

The ESP32 will collect sensor readings at predefined intervals and process the measurements for local visualization and cloud transmission. An OLED display will present live sensor readings and MQTT connection status, while an RGB LED will communicate predefined plant-care priority levels.

The communication subsystem will use Wi-Fi and a secure MQTT broker to exchange information between the ESP32 and the cloud platform. Transport Layer Security (TLS) will be used to protect communication, and structured JSON payloads will be used to represent sensor measurements and system messages.

The cloud backend will subscribe to the relevant MQTT topics, receive sensor measurements, and store the collected data with timestamps. It will also integrate a real weather API, an authorized email or notification account, and an LLM-based processing framework to generate contextual plant-care messages and summaries.

A Node-RED dashboard will provide centralized visualization of environmental measurements, connection information, MQTT messages, and AI-generated responses.

The overall architecture will support bidirectional communication, allowing the IoT device to publish sensor information and receive messages generated by the cloud backend. AI-generated messages will be available through MQTT and the dashboard but will not be displayed on the ESP32 OLED screen.

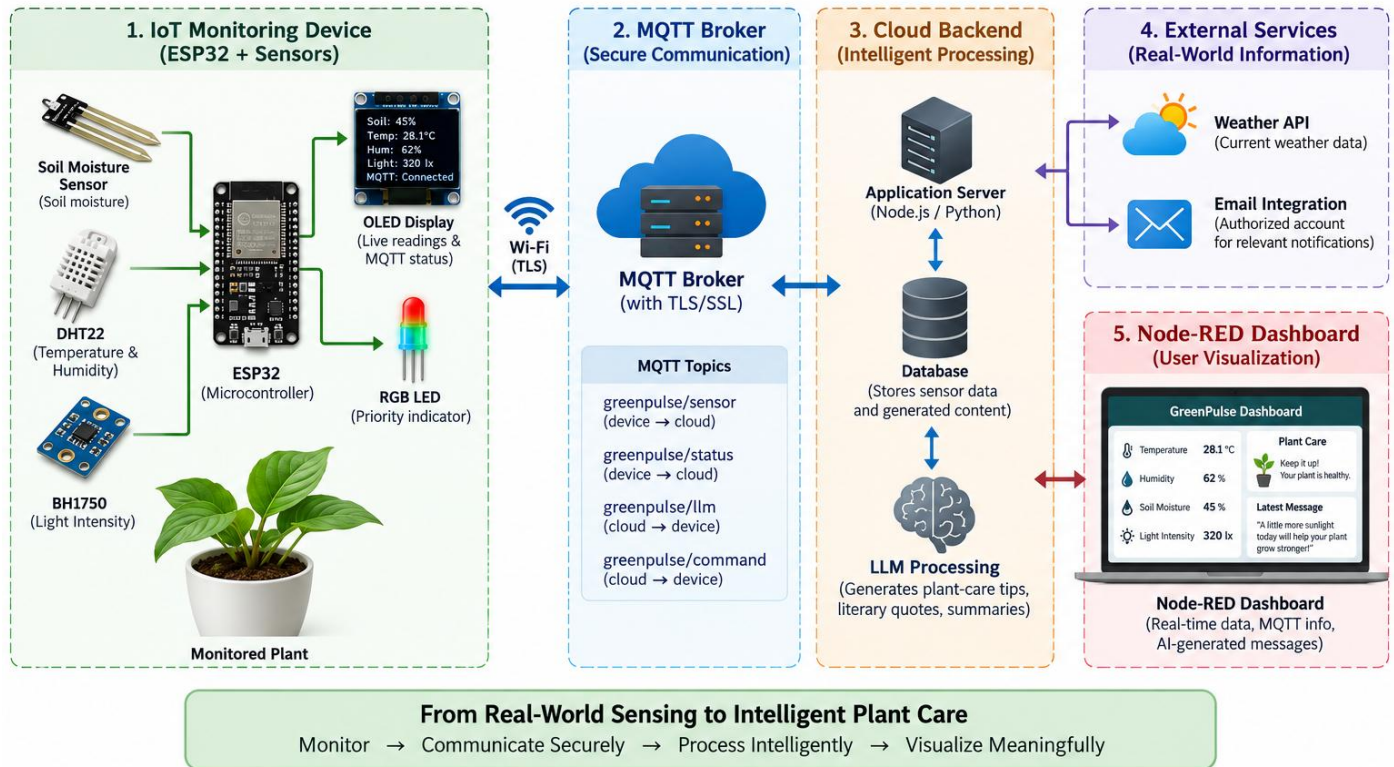


Figure 1- Proposed Overall System Architecture of GreenPulse

2.2 Overall System Architecture

The proposed GreenPulse architecture will follow a layered design to separate environmental monitoring, communication, intelligent processing, and visualization responsibilities.

2.2.1 Sensing Layer

The sensing layer will contain the environmental sensors connected to the ESP32 microcontroller. These sensors will collect measurements related to the plant's surrounding environment and soil conditions.

The soil moisture sensor will provide information about the moisture condition of the soil. The DHT22 sensor will measure ambient temperature and relative humidity, while the BH1750 sensor will provide ambient light intensity measurements. The collected measurements will form the primary input data for local status evaluation and cloud-based processing.

2.2.2 Device Processing Layer

The device processing layer will be implemented using the ESP32 microcontroller. The ESP32 will initialize the connected sensors, collect measurements, validate sensor readings, update the OLED display, and control the RGB LED according to predefined priority conditions. It will also establish Wi-Fi connectivity, connect to the MQTT broker, and publish sensor measurements using structured JSON messages.

2.2.3 Communication Layer

The communication layer will provide secure data exchange between the IoT device and the cloud platform. The ESP32 will communicate with the selected MQTT broker through Wi-Fi using TLS. MQTT topics will be defined to separate sensor telemetry, device status, and cloud-generated responses. The proposed design will support both publishing and subscribing, enabling bidirectional communication between the device and cloud backend.

2.2.4 Cloud Intelligence Layer

The cloud intelligence layer will receive sensor measurements through MQTT and perform data storage and intelligent processing. The backend will retrieve current weather information through a real weather API and relevant messages through an authorized email or notification account. An LLM-based framework will use the available information to generate plant-condition-inspired literary quotes, contextual care tips, weather summaries, and notification summaries. The generated information will be published to designated MQTT topics.

2.2.5 Visualization Layer

The visualization layer will be implemented using Node-RED. The dashboard will display live sensor measurements, MQTT connection status, topic names, timestamps, LLM input and output payloads, and generated plant-care information. This layer will allow users to observe the operation of the proposed system through a centralized interface.

2.3 Hardware Components and Sensor Selection

The proposed hardware design will use an ESP32 microcontroller as the central processing and communication unit. Environmental sensors will be connected to the microcontroller to obtain measurements, while an OLED display and RGB LED will provide local information and status indication.

The selected components are intended to support the environmental monitoring, connectivity, and local visualization requirements of the proposed system.

Table 1- Proposed Hardware Components and Their Functions

Component	Proposed Function
ESP32 microcontroller	Collect sensor measurements, execute firmware, manage Wi-Fi and MQTT communication, and control output devices.
Soil moisture sensor	Measure soil moisture conditions for plant monitoring.
DHT22 sensor	Measure ambient temperature and relative humidity.
BH1750 sensor	Measure ambient light intensity.
OLED display	Display live sensor readings and MQTT connection status.
RGB LED	Provide a visual indication of predefined plant-care priority levels.
Breadboard or PCB	Support prototype assembly and electrical connections.
Jumper wires and connectors	Establish electrical connections between components.
Regulated power supply	Provide suitable operating power to the ESP32 and connected components.

2.3.1 ESP32 Microcontroller

The ESP32 will function as the central controller of the GreenPulse monitoring device.

It will be responsible for collecting environmental measurements, executing sensor-processing logic, updating local output devices, and managing wireless communication. The ESP32 is proposed because it provides integrated Wi-Fi connectivity, general-purpose input/output pins, analog input capability, and support for common digital sensor interfaces. The firmware will be developed using an ESP32-compatible development environment and will coordinate sensor acquisition, local status evaluation, and secure MQTT communication.

2.3.2 Soil Moisture Sensor

A soil moisture sensor will be used to measure the moisture condition of the plant's soil.

The sensor will provide measurements that can be used to identify relatively dry or wet soil conditions. Its output will be read by the ESP32 and converted into a meaningful moisture representation through calibration. The calibration process will be planned around measurements obtained under selected dry and wet reference conditions. The resulting soil moisture information will be included in the MQTT telemetry payload and used by the cloud backend as an input for plant-care message generation. The exact sensor model and its electrical interface will be confirmed during hardware finalization.

2.3.3 DHT22 Temperature and Humidity Sensor

The DHT22 sensor will be used to measure ambient temperature and relative humidity near the monitored plant. Temperature and humidity measurements will provide information about the surrounding environment and will be transmitted to the cloud backend together with the other sensor readings. The ESP32 will communicate with the DHT22 through a suitable digital interface and collect measurements at intervals compatible with the sensor's operating requirements. The collected data will be presented on the OLED display and Node-RED dashboard and included in the structured input supplied to the LLM backend.

2.3.4 BH1750 Light Intensity Sensor

The BH1750 sensor will be used to measure ambient light intensity. Light intensity information will allow GreenPulse to monitor an additional environmental condition associated with plant maintenance. The BH1750 will communicate with the ESP32 through the I²C interface. Its measurements will be represented in lux and included in the sensor telemetry payload. The cloud backend will use the collected light measurements as part of the environmental context provided to the LLM.

2.3.5 OLED Display

An OLED display will be integrated into the device to provide immediate access to environmental measurements and MQTT connection status. The proposed display will present soil moisture, temperature, humidity, and light intensity readings using a compact layout. Where necessary, the readings will be arranged across multiple screens or displayed using a rotating information layout. The OLED will also provide a visible indication of the MQTT connection state. In accordance with the assignment requirements, the display will not present AI-generated care messages, literary quotes, MQTT payloads, or notification summaries.

2.3.6 RGB LED Priority Indicator

An RGB LED will be used to communicate the plant-care priority level through predefined colors.

The proposed indicator will distinguish between normal, warning, and critical conditions. The final threshold values will be selected and validated during prototype development. The priority logic will evaluate the available sensor measurements and determine the appropriate visual indication. The indicator will provide a simple local representation of the monitoring status without requiring the user to interpret every sensor measurement individually.

2.4 Proposed Circuit Design and GPIO Configuration

The GreenPulse hardware circuit will be designed around the ESP32 microcontroller.

Each sensor and output component will be connected to an appropriate ESP32 interface according to its electrical and communication requirements. The proposed circuit will include analog input connections for the selected soil moisture sensor where applicable, a digital connection for the DHT22 sensor, and I²C connections for the BH1750 sensor and OLED display. The RGB LED will be connected to suitable GPIO output pins through appropriate current-limiting circuitry. A regulated power supply will be selected to meet the voltage and current requirements of the complete prototype. The circuit design will also consider common grounding, suitable logic levels, stable sensor operation, and safe electrical connections.

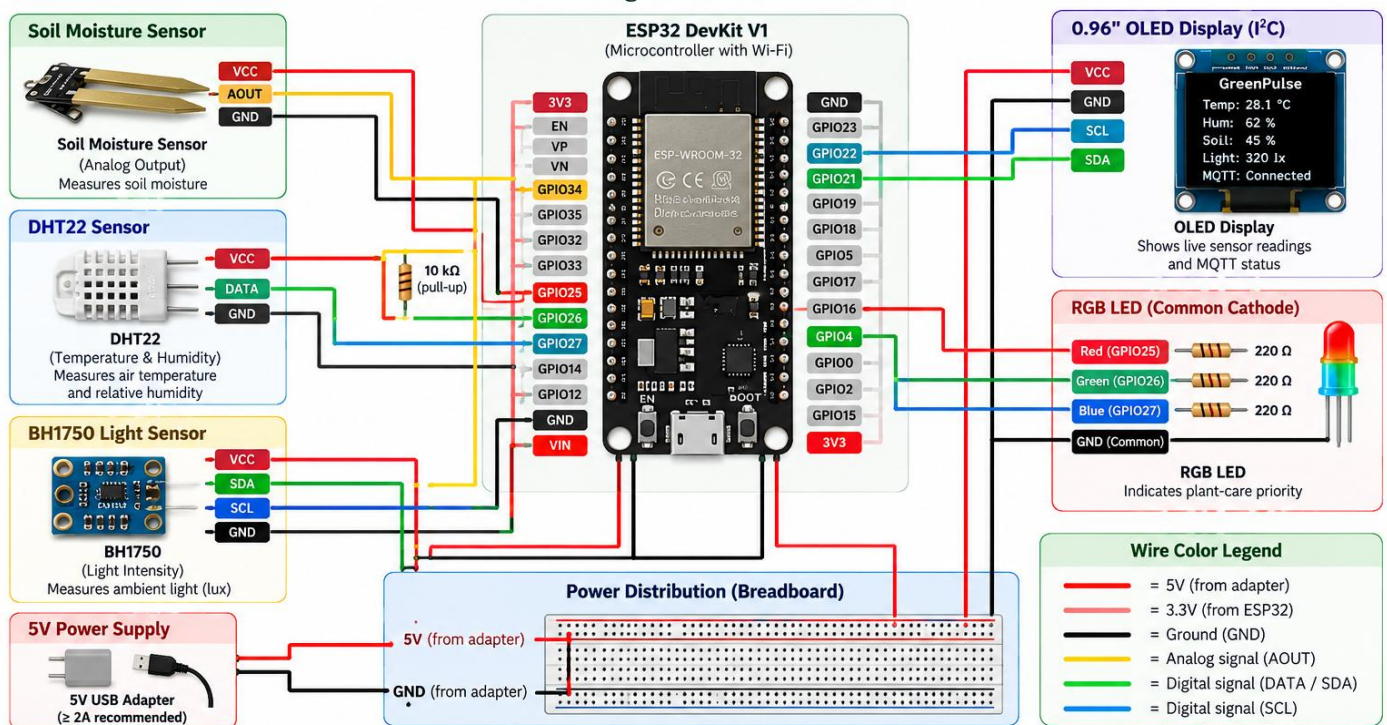


Figure 2 - : Proposed GreenPulse Hardware Circuit Diagram

Table 2- : Preliminary ESP32 GPIO Configuration

Component	Interface	Proposed ESP32 Connection
Soil moisture sensor	Analog, if applicable	GPIO34
DHT22	Digital	GPIO4
BH1750	I ² C SDA	GPIO21
BH1750	I ² C SCL	GPIO22
OLED display	I ² C SDA	GPIO21
OLED display	I ² C SCL	GPIO22
RGB LED — Red	Digital output	GPIO25
RGB LED — Green	Digital output	GPIO26
RGB LED — Blue	Digital output	GPIO27

The BH1750 and OLED display are proposed to share the I²C communication bus, provided that their configured addresses are compatible. The final circuit diagram will identify all power connections, signal connections, required resistors, and supporting components.

2.5 ESP32 Firmware Design

The ESP32 firmware will coordinate sensor acquisition, local output control, and communication with the cloud platform. The firmware will be organized into functional modules to support maintainability, testing, and integration.

2.5.1 System Initialization

During startup, the ESP32 will initialize serial communication, GPIO pins, connected sensors, the OLED display, and the RGB LED. The firmware will then attempt to establish Wi-Fi connectivity and connect securely to the selected MQTT broker. The current MQTT connection state will be made available through the OLED display.

2.5.2 Sensor Data Acquisition

The firmware will collect sensor measurements at predefined intervals. Soil moisture, temperature, humidity, and light intensity values will be obtained through their respective interfaces. Basic validation will be applied to identify failed sensor reads or invalid measurements. The sampling intervals will be selected according to sensor operating requirements and the intended monitoring frequency.

2.5.3 Local Display and Priority Processing

Following data acquisition, the ESP32 will update the OLED display with the latest valid measurements and MQTT connection status. The firmware will evaluate predefined sensor conditions to determine the appropriate priority level for the RGB LED. Critical conditions will take precedence over warning and normal conditions.

2.5.4 MQTT Data Publication

The ESP32 will format the collected measurements into structured JSON messages. The messages will contain relevant sensor values and device information and will be published to predefined MQTT topics through the secure connection. The exact payload structure and topic hierarchy will be documented in Chapter 3.

2.5.5 MQTT Subscription and Connection Management

The firmware will subscribe to designated topics for cloud-generated messages. Incoming messages will be received and processed according to the defined communication protocol. AI-generated text will not be rendered on the OLED display. The firmware will include reconnection procedures for temporary Wi-Fi or MQTT connection failures.

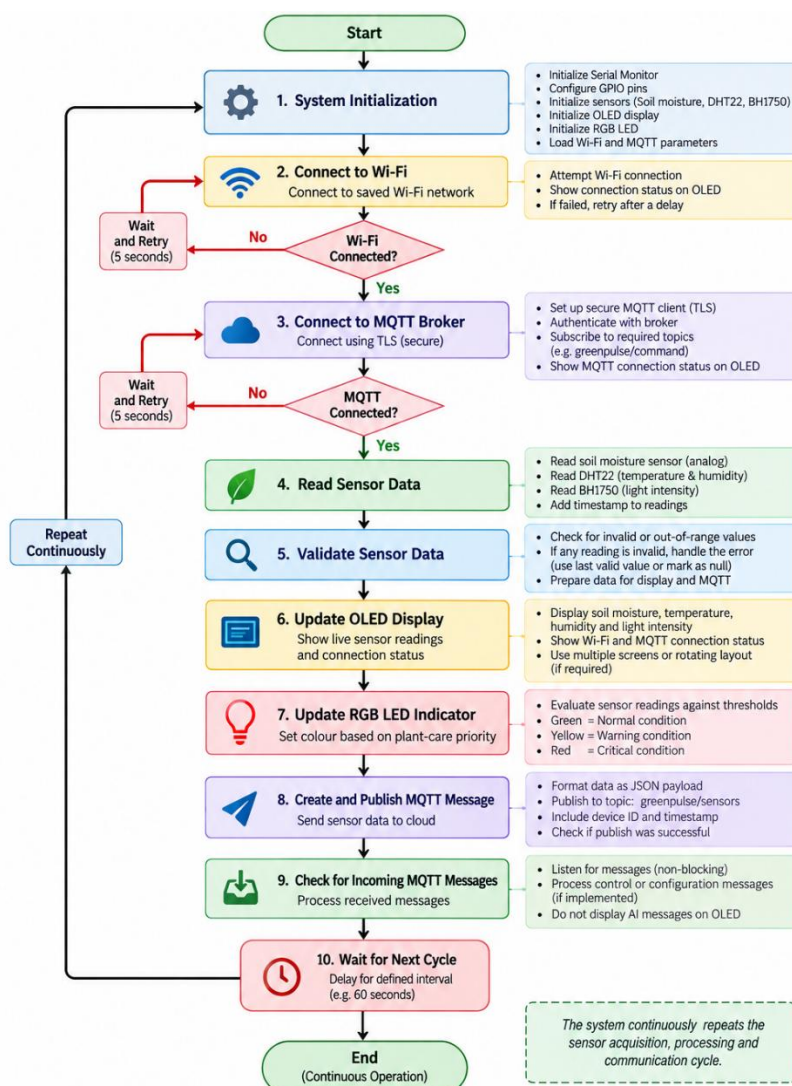


Figure 3- Proposed Firmware Operation and Data Acquisition Flow

2.6 OLED Display and Priority Indicator Design

The local user interface will consist of an OLED display and an RGB LED priority indicator. These components will provide immediate information about the monitored environment and device operation.

2.6.1 OLED Display Layout

The OLED display will be designed to show the latest available environmental measurements and MQTT connection status. A proposed layout is presented below.

Table 3- Proposed OLED Display Information

Display Item	Information
Soil Moisture	Current soil moisture reading
Temperature	Ambient temperature in °C
Humidity	Relative humidity in %
Light Intensity	Ambient light intensity in lux
MQTT Status	Connected or Disconnected

The final screen arrangement will depend on the selected OLED resolution and the amount of information that can be displayed legibly. If all measurements cannot be displayed simultaneously, the firmware will use multiple pages or a rotating display layout. The OLED will remain restricted to live sensor readings and MQTT connection status.

2.6.2 Priority Indicator Logic

The RGB LED will provide a color-based indication of plant-care priority.

The proposed priority model will consist of three levels.

Table 4- Proposed RGB LED Priority States

Priority Level	LED Color	Proposed Meaning
Normal	Green	Monitored conditions are within the selected acceptable ranges.
Warning	Yellow	One or more measurements require attention.
Critical	Red	One or more measurements exceed a defined critical threshold.

The final threshold values will be established through sensor calibration, selected plant requirements, and prototype testing. The system will apply a priority-based decision process in which critical conditions override warning conditions, and warning conditions override normal conditions. The selected priority state will be updated whenever new sensor measurements are processed.

2.6.3 Local Interface Design Considerations

The local interface will be designed to provide information without unnecessary complexity. The OLED will present numerical sensor measurements and connectivity information, while the RGB LED will provide an immediate visual representation of the priority state. AI-generated care tips, literary quotes, weather summaries, notification summaries, and MQTT payload information will be made available through the communication and dashboard layers rather than the local display. This separation will ensure that the proposed device interface remains consistent with the assignment requirements while allowing the cloud and dashboard components to provide more detailed information.

CHAPTER 3: COMMUNICATION AND CLOUD ARCHITECTURE

3.1 Secure MQTT Communication

Secure communication is an essential requirement of the proposed GreenPulse system because environmental measurements and cloud-generated information will be exchanged between the ESP32 monitoring device and a remotely hosted backend. The communication architecture will be designed to support reliable data transmission, bidirectional messaging, and protection against unauthorized access.

GreenPulse will use the Message Queuing Telemetry Transport (MQTT) protocol as its primary communication mechanism. MQTT is a lightweight publish-subscribe messaging protocol that is suitable for IoT applications because it enables devices and applications to exchange messages through a centralized broker without requiring direct communication between individual clients.

The proposed system will use AWS IoT Core as its MQTT broker. The ESP32, cloud backend, and Node-RED dashboard will connect to the broker as MQTT clients, with each component subscribing to or publishing information through designated topics.

3.1.1 Wi-Fi Connectivity

The ESP32 will use its integrated Wi-Fi capability to establish an internet connection through an available wireless network. During initialization, the firmware will attempt to connect to the configured Wi-Fi network. Once connectivity is established, the device will initiate a secure connection to the MQTT broker.

The firmware will monitor connectivity and attempt reconnection when the Wi-Fi or MQTT connection is interrupted. The OLED display will show the current MQTT connection status, allowing users to identify whether the device is connected to the communication infrastructure.

3.1.2 MQTT Broker Selection

AWS IoT Core is proposed as the MQTT communication platform for GreenPulse. The broker will act as an intermediary between the monitoring device, cloud backend, and dashboard. This approach will allow the different system components to exchange information through defined topics without maintaining direct connections to one another. The ESP32 will publish sensor measurements and device status messages, while the cloud backend will subscribe to the relevant telemetry topics.

After processing the received information, the backend will publish AI-generated responses to separate topics. The ESP32 and Node-RED dashboard will subscribe to the topics relevant to their respective functions. The use of a centralized MQTT broker will support modular development and simplify communication between the proposed subsystems.

3.1.3 Transport Layer Security

Transport Layer Security (TLS) will be implemented to protect MQTT communication between the system components and AWS IoT Core. The proposed ESP32 connection will use MQTT over TLS through port 8883, subject to the selected endpoint and supported authentication configuration. AWS IoT Core device certificates and associated private keys will be used to authenticate the ESP32. The device will also use the appropriate trusted certificate authority information to validate the server connection.

AWS IoT policies will be configured to restrict the device to the MQTT actions and topics required for its operation. Sensitive credentials and private keys will not be included in the public source-code repository. The cloud backend and Node-RED dashboard will use authentication methods compatible with their selected deployment environments and AWS IoT Core connection configurations.

3.1.4 Communication Reliability

The communication design will include mechanisms for handling temporary connection failures. The ESP32 firmware will monitor the Wi-Fi and MQTT connection states and initiate reconnection attempts when necessary.

Sensor acquisition and local status indication will be designed to continue when cloud connectivity is temporarily unavailable, where practical. MQTT Quality of Service (QoS) level 1 is proposed for important sensor telemetry and AI response messages to support at-least-once delivery. The backend will therefore be designed to tolerate duplicate messages. The final QoS settings will be verified against the capabilities of the selected MQTT clients and broker.

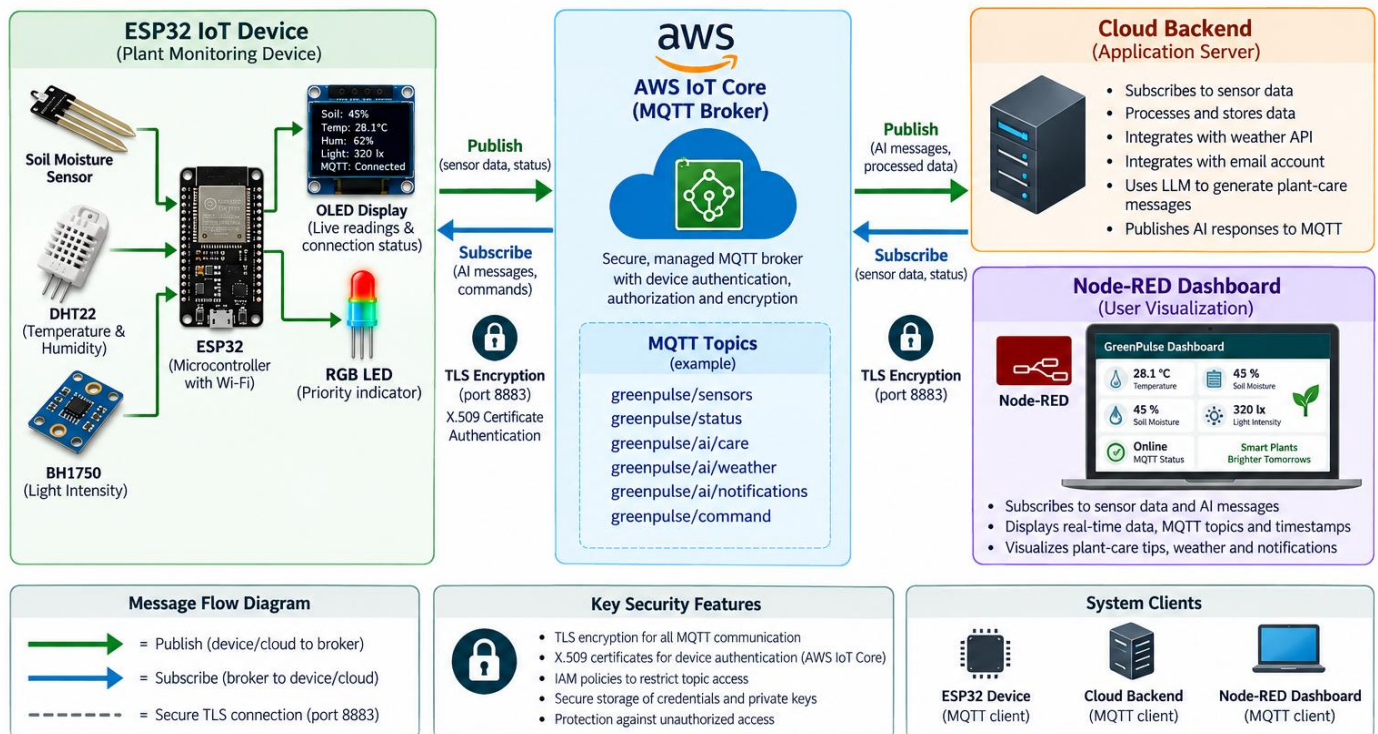


Figure 4 - Proposed Secure MQTT Communication Architecture of GreenPulse

3.2 MQTT Topics and JSON Payloads

The proposed GreenPulse system will use a structured MQTT topic hierarchy to separate sensor telemetry, device status, and AI-generated information. Each topic will have a defined purpose, publisher, and subscriber. This organization will improve maintainability and allow the Node-RED dashboard to subscribe to the information required for visualization.

The proposed topic hierarchy will use the prefix `greenpulse/` followed by the message category.

Table 5 - Proposed MQTT Topics and Message Directions

MQTT Topic	Publisher	Subscriber	Purpose
<code>greenpulse/sensors</code>	ESP32	Cloud backend, Node-RED	Publish environmental sensor measurements.
<code>greenpulse/status</code>	ESP32	Cloud backend, Node-RED	Publish device and connectivity status.
<code>greenpulse/ai/care</code>	Cloud backend	ESP32, Node-RED	Publish LLM-generated plant-care messages and literary quotes.
<code>greenpulse/ai/weather</code>	Cloud backend	ESP32, Node-RED	Publish weather-related summaries.
<code>greenpulse/ai/notifications</code>	Cloud backend	ESP32, Node-RED	Publish summaries of relevant email or notification information.
<code>greenpulse/command</code>	Cloud backend	ESP32	Deliver defined device commands or configuration messages if implemented.

The topics in Table 3.1 represent the proposed communication structure and may be refined during implementation. Although the ESP32 will subscribe to selected cloud-generated topics to demonstrate bidirectional communication, AI-generated text and MQTT payloads will not be displayed on its OLED screen.

3.2.1 Sensor Telemetry Payload

Environmental sensor measurements will be formatted as JSON messages before transmission. The payload will contain a device identifier, measurement values, and information required to associate the readings with a particular acquisition cycle.

A proposed sensor telemetry payload is presented below.

Example 3.1: Proposed Sensor Telemetry JSON Payload

```
{
  "device_id": "greenpulse_01",
  "sequence_number": 125,
  "soil_moisture": 45,
  "temperature": 28.1,
  "humidity": 62,
  "light_intensity": 320
}
```

The values in Example 3.1 are illustrative and do not represent actual experimental measurements. The cloud backend will associate received messages with server-side timestamps. Where the ESP32 has access to a synchronized clock, a device-generated timestamp may also be included. The final telemetry structure will

be expanded to include measurements from any additional sensors retained from the original GreenPulse design.

3.2.2 Device Status Payload

A separate MQTT topic will be used to communicate the operational status of the monitoring device.

The status payload may contain the device identifier, firmware version, connection information, and the current priority state.

Example 3.2: Proposed Device Status JSON Payload

```
{
  "device_id": "greenpulse_01",
  "firmware_version": "1.0.0",
  "priority": "normal",
  "wifi_connected": true,
  "mqtt_connected": true
}
```

The proposed status message will help the backend and dashboard identify the reported operating condition of the device.

3.2.3 AI-Generated Care Message Payload

The cloud backend will publish the LLM-generated care message to a designated MQTT topic. The message will contain the device identifier, generated text, and a timestamp indicating when the response was produced.

Example 3.3: Proposed AI Care Message Payload

```
{
  "device_id": "greenpulse_01",
  "timestamp": "2026-09-17T10:30:00Z",
  "priority": "normal",
  "care_tip": "Monitor the soil moisture before the next watering.",
  "literary_quote": "Every leaf tells a story of the care it receives."
}
```

The text and timestamp in Example 3.3 are illustrative examples of the proposed message format. The Node-RED dashboard will display the generated information. The ESP32 may receive the message through its MQTT subscription, but it will not render the AI-generated text on the OLED display.

3.2.4 Weather and Notification Payloads

Weather and notification summaries will be published through separate MQTT topics. The weather message will contain a generated summary based on information retrieved from a real weather API. The notification message will contain a summary of relevant information obtained from an authorized email or notification account. Separating these messages will allow the dashboard to display each information category independently and simplify debugging during system integration.

3.2.5 Payload Validation

The cloud backend will validate incoming JSON messages before processing or storing them. Validation will include checking the required fields, supported data types, device identifiers, and the presence of expected sensor measurements. Messages containing invalid or incomplete information will be handled according to predefined error-processing rules. This approach will reduce the risk of malformed data affecting cloud storage, dashboard visualization, or LLM processing.

3.3 Cloud Backend Architecture

The GreenPulse cloud backend will be responsible for receiving sensor measurements, storing environmental data, retrieving external information, coordinating LLM processing, and publishing generated responses.

A Python-based backend is proposed because Python provides libraries and frameworks suitable for MQTT communication, API integration, data processing, and LLM application development.

The backend will be deployed on a cloud-hosted computing environment with access to the selected MQTT broker, database, weather API, email integration, and LLM service.

The final deployment service will be selected according to the group's implementation requirements, available resources, and cost considerations.

3.3.1 MQTT Subscriber Service

The backend will establish an authenticated MQTT connection and subscribe to the sensor telemetry and device status topics. When a sensor message is received, the backend will parse the JSON payload and validate its contents.

Valid measurements will be forwarded to the data storage component and made available for intelligent processing. The subscriber will be designed to handle incoming messages without unnecessarily blocking subsequent MQTT communication.

3.3.2 Sensor Data Processing

The data processing component will organize incoming sensor measurements into a consistent internal structure. It will identify the originating device, associate measurements with timestamps, and check the availability of the required data fields.

The processed information will be used for database storage and preparation of structured LLM input. The backend may also apply basic threshold evaluation and identify missing or invalid measurements before invoking the LLM.

3.3.3 External Service Integration

The cloud backend will integrate a real weather API to obtain current weather information relevant to the monitored plant's location. The proposed weather integration will retrieve fields such as ambient temperature, humidity, rainfall information, and other available conditions relevant to the selected API.

The backend will also connect to an authorized Gmail, Outlook, or IMAP account to retrieve relevant email or notification information. Authentication credentials and access tokens will be managed securely and will not be exposed through public source code or MQTT messages. The detailed weather and notification processing workflows will be presented in Chapter 4.

3.3.4 LLM Processing Service

The backend will prepare structured input for the LLM using the latest valid sensor measurements and the relevant external information. An LLM framework will be used to organize the processing workflow and generate plant-condition-inspired literary quotes, contextual care tips, weather summaries, and notification summaries.

The generated output will be checked for the expected structure before publication. The LLM will be instructed to use the supplied measurements and retrieved information without inventing unavailable sensor readings, weather conditions, or notification contents.

3.3.5 MQTT Response Publisher

Following successful processing, the backend will publish the generated responses to their respective MQTT topics. Care tips, weather summaries, and notification summaries will be transmitted as structured JSON messages.

The Node-RED dashboard will subscribe to these topics and display the generated information. The ESP32 will subscribe to the selected cloud response topics as required for bidirectional communication, while maintaining the assignment's restriction on OLED content.

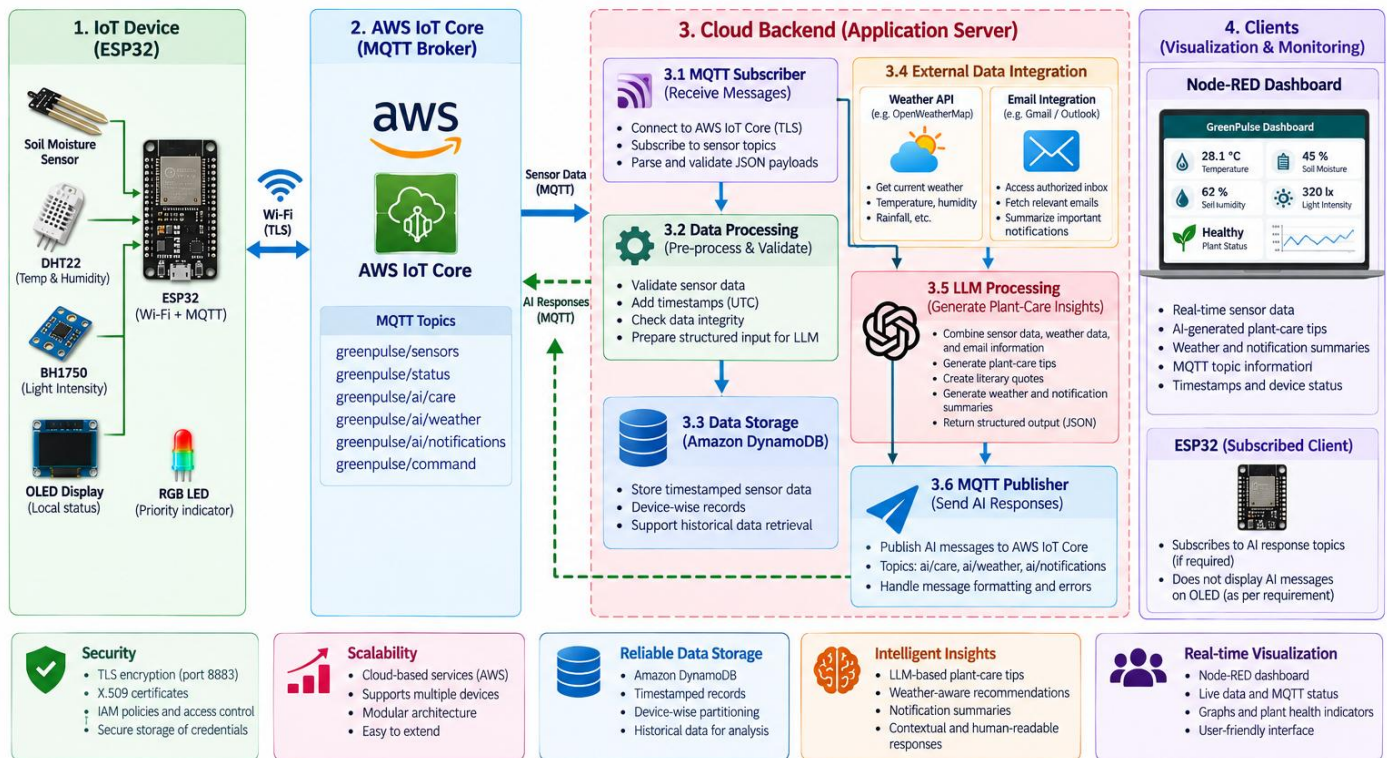


Figure 5 - Proposed Cloud Backend and Data Processing Architecture of GreenPulse

3.4 Data Storage Design

A cloud-based data storage mechanism will be incorporated into GreenPulse to maintain a timestamped record of environmental measurements.

Amazon DynamoDB is proposed as the primary storage service because it can store structured records and integrate with applications deployed within the AWS environment. The database will support the retrieval of historical sensor information for monitoring, troubleshooting, and possible future extensions.

3.4.1 Sensor Data Collection

When the backend receives a valid MQTT telemetry message, it will create a database record containing the device identifier, timestamp, and sensor measurements.

The database will store the received environmental data rather than relying exclusively on temporary values held in application memory. Each record will represent a sensor acquisition event.

3.4.2 Proposed Database Structure

The database will use a device identifier and timestamp to organize sensor records. A composite key consisting of device_id as the partition key and timestamp as the sort key is proposed for the sensor telemetry table.

Table 6 - Proposed Sensor Data Storage Schema

Field	Data Type	Description
device_id	String	Unique identifier of the ESP32 device.
timestamp	String	Timestamp associated with the received sensor record.
sequence_number	Number	Sequence identifier for the acquisition cycle.
soil_moisture	Number	Soil moisture measurement.
temperature	Number	Ambient temperature in degrees Celsius.
humidity	Number	Relative humidity percentage.
light_intensity	Number	Ambient light intensity in lux.
priority	String	Evaluated plant-care priority, where available.

The schema will be updated to accommodate additional sensors included in the finalized hardware design. Timestamps will use a consistent representation, such as ISO 8601 in UTC, to support chronological ordering and interpretation.

3.4.3 Data Storage and Retrieval

The backend will insert validated sensor records into the database after receiving telemetry messages. Stored records will be retrievable using the device identifier and timestamp.

This design will support access to historical environmental measurements and provide a foundation for future analytical features. Database access permissions will be restricted to the application components that require them.

3.4.4 Data Management Considerations

The proposed storage design will consider data consistency, duplicate messages, retention, and access control. Because MQTT QoS level 1 may result in duplicate message delivery, the backend will use device identifiers, sequence numbers, or another suitable deduplication strategy to reduce unintended duplicate records. The system will also define an appropriate retention approach to control storage usage and cloud costs.

3.5 Bidirectional Communication

Bidirectional MQTT communication will allow the ESP32 and cloud backend to exchange information through the selected broker. The proposed communication flow will include device-to-cloud telemetry and cloud-to-device response messages.

3.5.1 Device-to-Cloud Communication

The ESP32 will periodically collect environmental measurements and publish them to the sensor telemetry topic. The cloud backend will receive the messages, validate the payloads, and store the measurements.

The Node-RED dashboard will subscribe to the same telemetry topic to display the latest environmental information. The ESP32 will also publish device status messages to support connectivity monitoring.

3.5.2 Cloud-to-Device Communication

The cloud backend will generate contextual information using the LLM and publish the resulting messages to designated MQTT topics. The ESP32 will subscribe to the selected response topics and receive the published messages. Incoming messages may be acknowledged or processed according to the implemented firmware requirements. AI-generated text and MQTT payload contents will not be displayed on the OLED screen. The Node-RED dashboard will display the cloud-generated messages and their associated MQTT information.

3.5.3 End-to-End Communication Sequence

The proposed end-to-end communication process will follow these stages:

1. The ESP32 will initialize the connected sensors and establish Wi-Fi connectivity.
2. The device will connect securely to AWS IoT Core using MQTT over TLS.
3. The ESP32 will collect environmental measurements and prepare a structured JSON payload.
4. The device will publish the sensor measurements to the designated telemetry topic.
5. The cloud backend will receive, validate, and store the sensor information.
6. The backend will retrieve relevant weather and notification information when required.
7. The LLM processing service will generate contextual care messages and summaries.
8. The backend will publish the generated responses through designated MQTT topics.
9. The Node-RED dashboard will display sensor readings, connection information, MQTT payloads, and generated messages.
10. The ESP32 will continue monitoring and repeat the communication process at the configured interval.

This communication architecture will establish the foundation for integrating the physical monitoring device with the cloud-hosted intelligent processing system. The proposed design will support secure information exchange, structured message handling, timestamped data storage, and centralized visualization while maintaining the separation between the local device display and AI-generated content.

CHAPTER 4: INTELLIGENT SYSTEM DESIGN

4.1 LLM-Based Plant-Care Intelligence

The proposed GreenPulse system will incorporate a Large Language Model (LLM)-based intelligent processing component to transform environmental sensor measurements into contextual plant-care information. Unlike a conventional IoT monitoring system that primarily displays numerical measurements, GreenPulse will combine sensor data with external information to generate understandable care tips and plant-condition-inspired literary quotes.

The LLM processing component will operate within the cloud backend rather than on the ESP32 microcontroller. This separation will allow the physical monitoring device to focus on sensor acquisition, local visualization, priority indication, and MQTT communication while the cloud infrastructure handles computationally intensive language-processing tasks.

4.1.1 LLM Framework and Processing Architecture

A Python-based LLM processing framework, such as LangChain, is proposed to coordinate the intelligent processing workflow. The framework will connect the backend application to a selected LLM service and organize the preparation of input data, prompt construction, response generation, and output formatting.

The backend will subscribe to the sensor telemetry topic through AWS IoT Core. After receiving and validating the sensor measurements, it will organize the readings into a structured input format.

The LLM will receive the available environmental information and generate responses according to predefined instructions. The generated output will be validated and converted into JSON before publication through MQTT.

The final LLM model and hosting arrangement will be selected according to availability, integration requirements, response quality, and operational cost.

4.1.2 Sensor Data Preparation

The backend will prepare the latest valid sensor measurements before submitting them to the LLM.

The proposed input will include soil moisture, ambient temperature, relative humidity, and light intensity. Measurements from additional sensors in the finalized GreenPulse hardware design will also be incorporated.

The backend will identify missing or invalid measurements and prevent them from being presented to the LLM as valid observations.

The data preparation process will organize measurements with their units, timestamps, and relevant context. This structured representation will help the LLM distinguish actual measurements from generated interpretations.

4.1.3 Prompt Engineering

Prompt engineering will be used to define the LLM's role, input information, expected output structure, and response limitations.

The proposed prompt will instruct the LLM to generate a concise plant-care tip and a plant-condition-inspired literary quote based on the supplied environmental measurements.

The prompt will also instruct the model not to invent sensor measurements or claim that unavailable information has been observed.

A proposed prompt structure is presented below.

Example 4.1: Proposed LLM Prompt

System Instruction:

You are the GreenPulse plant-care assistant. Generate a concise plant-care tip and a short literary-style quote inspired by the supplied environmental measurements. Use only the available measurements when describing the monitored conditions. Do not invent sensor readings or make unsupported claims about the plant's health.

Input Data:

- Soil Moisture: 45%
- Temperature: 28.1°C
- Humidity: 62%
- Light Intensity: 320 lux

Required Output:

Return a structured JSON object containing a care tip and a literary quote.

The measurements in this example are illustrative and do not represent actual experimental results.

4.1.4 Plant-Care Tip Generation

The LLM will generate plant-care information using the available sensor measurements and relevant contextual data.

The generated tips will be designed to help users interpret environmental conditions and identify potential care requirements.

For example, when soil moisture is below a selected threshold, the system may generate a message advising the user to check whether watering is required.

The backend will use predefined thresholds and data-validation rules to provide additional structure to the decision-making process.

The LLM-generated information will be treated as contextual assistance rather than a guaranteed diagnosis of plant health.

4.1.5 Literary Quote Generation

In addition to practical care information, GreenPulse will generate short literary-style quotes inspired by the monitored plant's environmental conditions.

This functionality will demonstrate the creative language-generation capability of the LLM and provide a more engaging user experience.

The generated quotes will be published through the designated MQTT topic and displayed on the Node-RED dashboard.

4.1.6 LLM Output Validation and Publication

The backend will check the generated response before publishing it.

Validation will include verifying that the expected fields are present, the JSON structure is valid, and the response does not contain missing or malformed content.

When LLM processing fails, the backend will record the error and apply a predefined recovery procedure.

Successfully validated responses will be published to the greenpulse/ai/care topic.

The AI-generated content will be available through the MQTT communication layer and Node-RED dashboard. It will not be displayed on the ESP32 OLED screen.

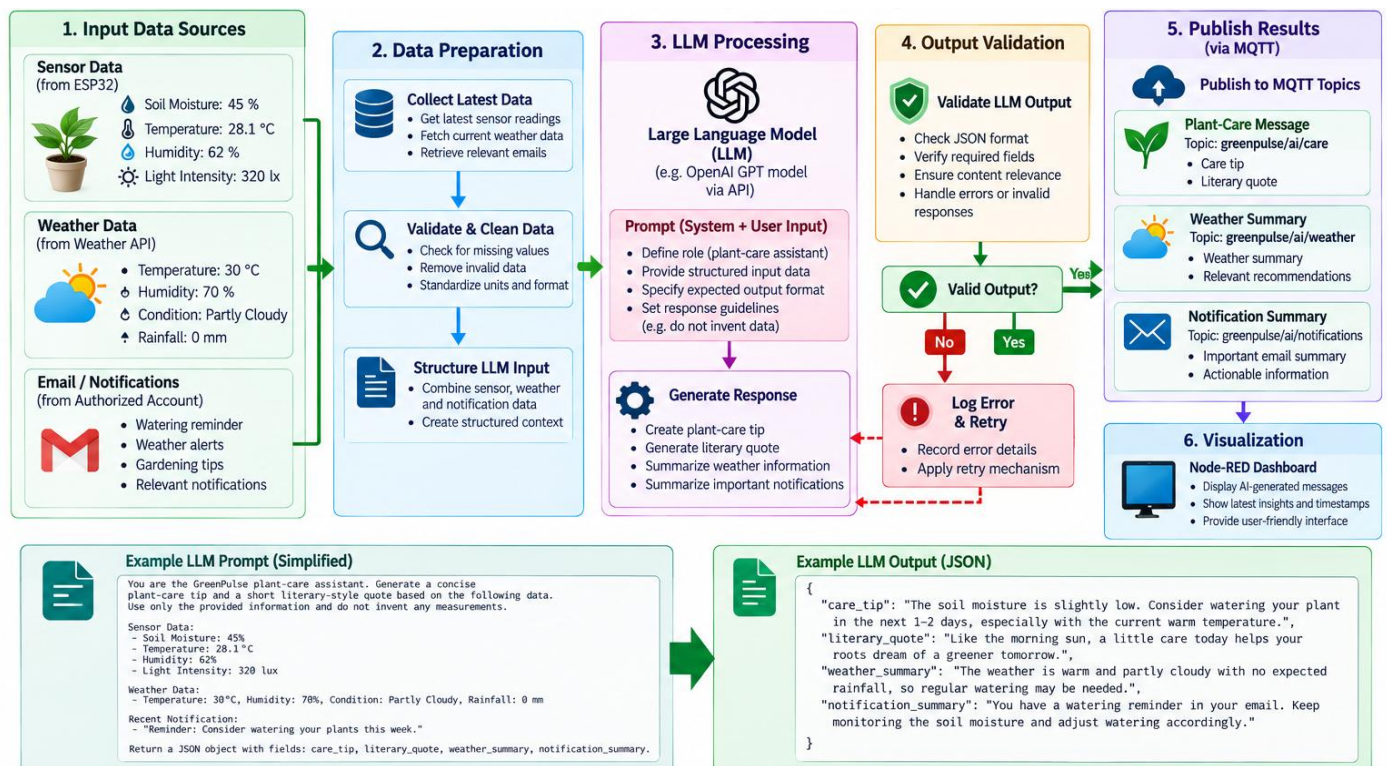


Figure 6 - Proposed LLM-Based Plant-Care Processing Workflow of GreenPulse

4.2 Real-Time Weather API Integration

GreenPulse will integrate a real weather API to retrieve current weather information relevant to the monitored plant's location.

Weather information will provide additional context beyond the measurements collected directly by the ESP32.

The proposed integration will support the generation of weather-aware plant-care summaries and demonstrate the use of external real-world information within the intelligent processing workflow.

4.2.1 Weather Data Source

A publicly accessible weather service, such as OpenWeatherMap, is proposed as the weather data provider.

The final API will be selected based on the availability of required weather information, integration capabilities, usage limits, and cost.

The backend will retrieve weather information for a configured location using the API's supported authentication and request methods.

API credentials will be stored securely in the backend environment rather than embedded in the ESP32 firmware or exposed through MQTT messages.

4.2.2 Weather Data Collection

The proposed weather integration will retrieve relevant fields such as:

- Current temperature.
- Relative humidity.
- Rainfall information, where available.
- General weather conditions.
- Other relevant fields supported by the selected API.

The backend will record the retrieval time and identify missing or unavailable weather fields.

Weather requests will be performed at a suitable interval to avoid unnecessary API usage and remain within the selected service's limits.

4.2.3 Weather-Aware LLM Processing

The retrieved weather information will be combined with the available environmental sensor measurements to prepare a structured LLM input.

The LLM will be instructed to summarize the actual weather information and explain its potential relevance to plant-care activities.

For example, the system may consider whether rainfall information or high ambient temperatures are relevant to the user's watering decisions.

The generated response will clearly distinguish between measured sensor conditions, retrieved weather information, and the model's interpretation.

4.2.4 Weather Summary Publication

After processing, the backend will publish the generated weather summary to the `greenpulse/ai/weather` MQTT topic.

The Node-RED dashboard will subscribe to this topic and display the summary together with the relevant timestamp.

The weather integration will include error handling for unsuccessful API requests, unavailable data, and temporary network interruptions.

4.3 Email and Notification Integration

The proposed GreenPulse system will integrate an authorized email or notification account to retrieve information relevant to watering, weather, and gardening activities.

This functionality will demonstrate how the cloud backend can combine IoT sensor measurements with information obtained from an external communication service.

The integration will use an actual account authorized for the project rather than relying exclusively on fabricated notification data.

4.3.1 Email Service Selection

The system will use a supported email integration method, such as the Gmail API, Microsoft Graph for Outlook, or an appropriately configured IMAP connection.

The final service will be selected according to the group's available account, authentication requirements, and implementation feasibility.

The integration will use an authorized account and request only the permissions necessary to retrieve the relevant information.

4.3.2 Notification Retrieval

The backend will periodically retrieve relevant messages from the configured account.

The proposed filtering process will identify messages associated with watering reminders, weather alerts, gardening activities, or other plant-care-related information.

Filtering may use message metadata, subject information, and relevant content, depending on the capabilities of the selected integration.

The system will avoid unnecessarily processing unrelated personal messages.

4.3.3 LLM-Based Notification Summarization

Relevant notification information will be organized into a structured input and submitted to the LLM. The LLM will generate concise summaries highlighting information that may be useful for plant maintenance.

The processing instructions will require the model to summarize the retrieved information without inventing messages, events, or instructions that are not present in the source material. The generated summaries will be prepared in a structured format for MQTT publication.

4.3.4 Privacy and Security

The email integration will be designed to protect account credentials and personal information. Authentication tokens and other sensitive credentials will be stored securely within the backend environment.

The system will use the minimum necessary access permissions and limit processing to information relevant to the project. Raw email content and sensitive account information will not be published to publicly accessible MQTT topics. Only the information required for the proposed notification-summary functionality will be included in the generated output.

4.3.5 Notification Summary Publication

Validated notification summaries will be published to the `greenpulse/ai/notifications` MQTT topic. The Node-RED dashboard will display the generated summaries and their associated timestamps. The backend will also handle authentication failures, unavailable messages, and temporary service interruptions.

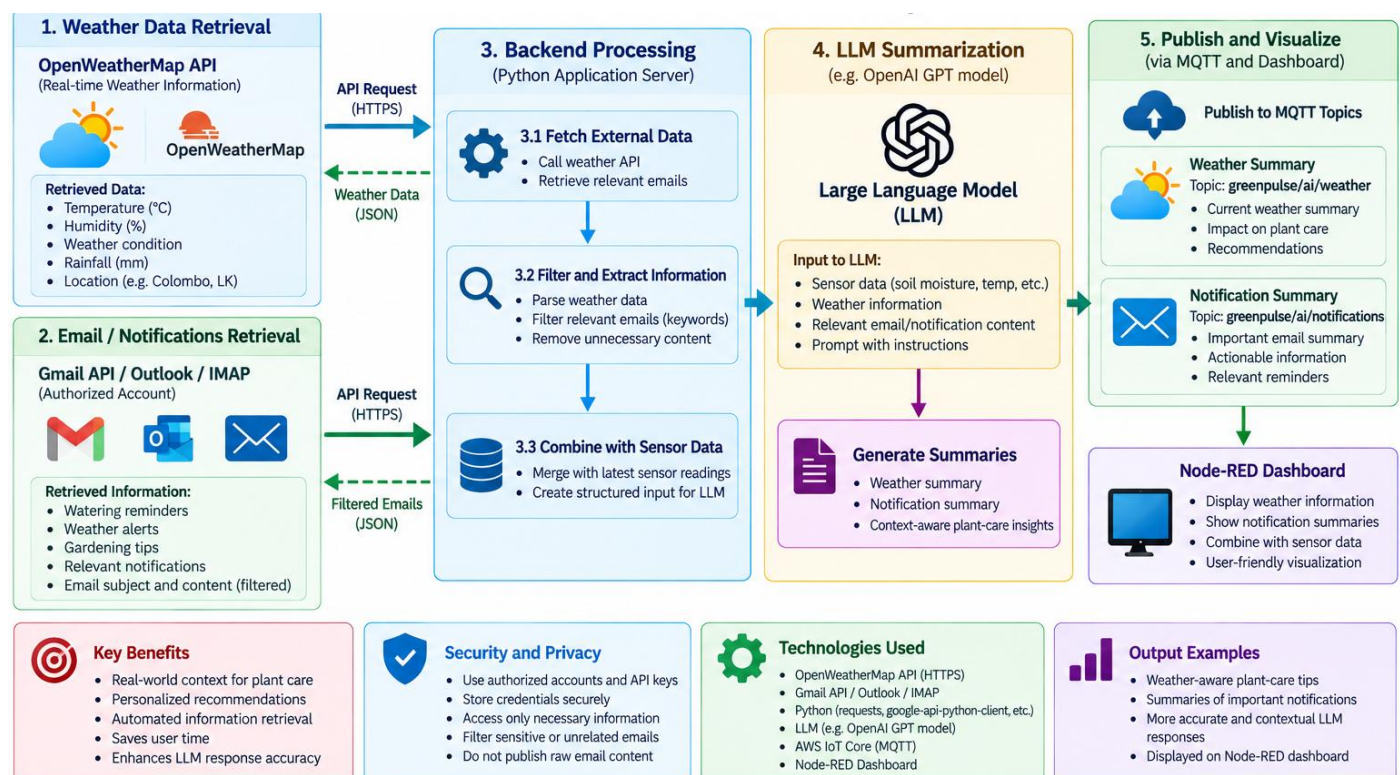


Figure 7 - Proposed External Information Retrieval and LLM Summarization Workflow

4.4 Node-RED Dashboard Design

A Node-RED dashboard will be developed to provide centralized visualization of the GreenPulse system. The dashboard will allow users to observe live environmental measurements, monitor communication status, inspect MQTT messages, and view AI-generated plant-care information.

Node-RED will connect to the selected MQTT broker using an appropriate authenticated and secure connection. The dashboard will subscribe to the topics required for sensor visualization and intelligent output monitoring.

4.4.1 Live Sensor Monitoring

The dashboard will display the latest available environmental measurements received from the ESP32. The proposed monitoring interface will include soil moisture, temperature, humidity, and light intensity values.

Additional sensor measurements will be included when the finalized hardware design is confirmed. The dashboard may use numerical widgets, gauges, and charts to present the information in an understandable format.

4.4.2 MQTT Communication Monitoring

The dashboard will display relevant MQTT topic names, message timestamps, and connection status. It will also provide visibility into incoming sensor telemetry and cloud-generated responses. This information will support the demonstration and troubleshooting of the proposed bidirectional communication architecture.

4.4.3 LLM Input and Output Visualization

The dashboard will provide dedicated sections for displaying the information supplied to the LLM and the generated output returned by the intelligent processing service. The LLM input section will display the structured environmental information and relevant contextual data used during processing.

The output section will display the generated care tips, literary quotes, weather summaries, and notification summaries. Sensitive information, including credentials and unnecessary personal email content, will be excluded from dashboard payloads.

4.4.4 Proposed Dashboard Layout

The proposed dashboard will be organized into the following functional sections.

Table 7 - Proposed Node-RED Dashboard Components

Dashboard Section	Proposed Content
Environmental Monitoring	Live soil moisture, temperature, humidity, and light intensity.
Device Status	Device identifier, MQTT connection status, and priority level.
MQTT Monitoring	Topic names, timestamps, and received message information.
LLM Input	Structured information submitted for intelligent processing.

AI Plant Care	Generated care tips and literary quotes.
Weather Summary	LLM-generated summary of retrieved weather information.
Notification Summary	Summary of relevant information from the authorized email account.

The dashboard layout will be designed to present the required information clearly without unnecessarily exposing sensitive data.

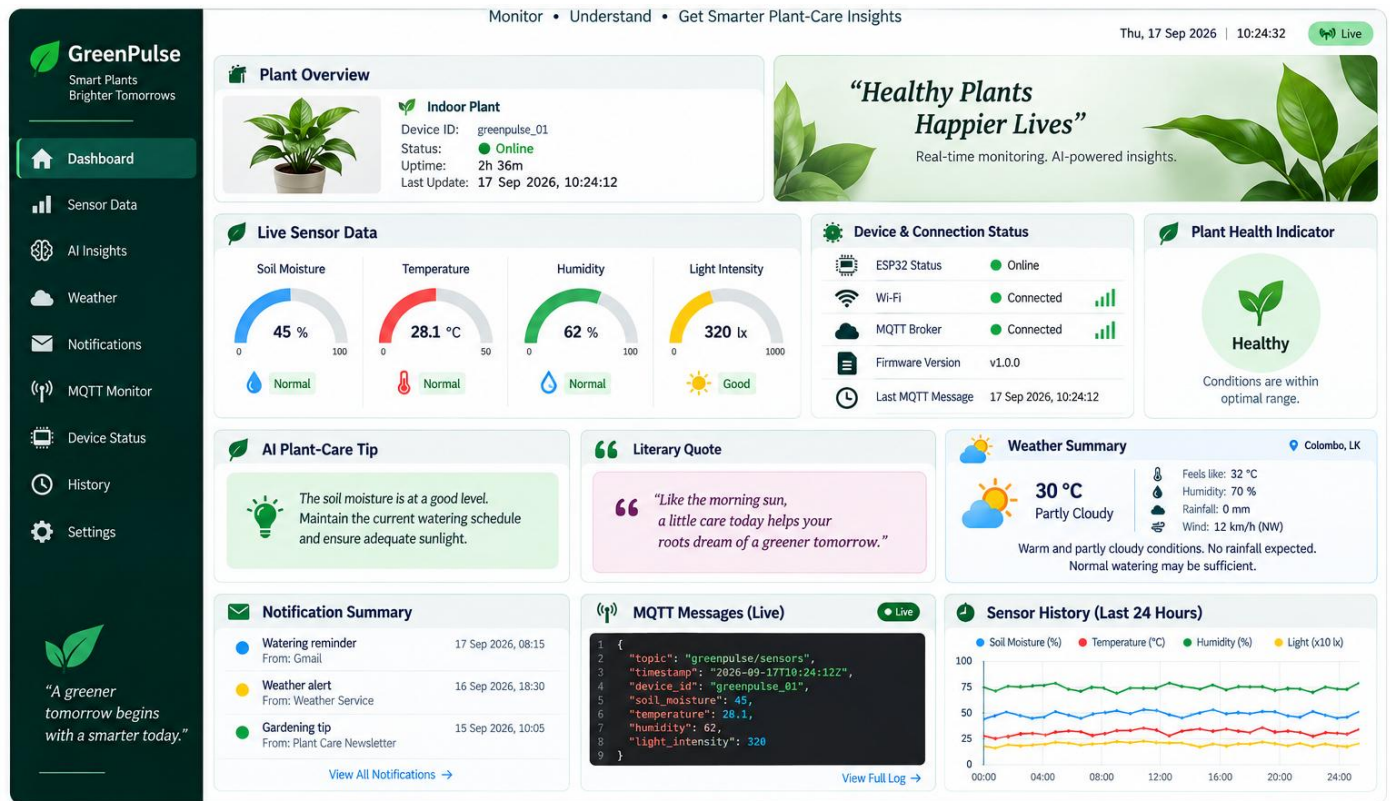


Figure 8 - Proposed Node-RED Dashboard Interface for GreenPulse

4.5 Proposed Advanced Features

GreenPulse will investigate additional functionality beyond the minimum environmental monitoring and LLM integration requirements. The final selection of advanced features will depend on hardware availability, development time, technical feasibility, and the group's implementation priorities.

4.5.1 Additional Environmental Sensors

The proposed system will retain the additional sensors included in the original GreenPulse hardware design. These sensors will expand the environmental information available to the ESP32 and cloud backend. Their measurements will be incorporated into the firmware, MQTT telemetry structure, data storage schema, dashboard, and relevant LLM inputs.

4.5.2 Data-Driven Watering Predictions

A potential extension will involve using current and historical soil moisture measurements to estimate when the monitored plant may require watering. This functionality will require the collection of sufficient timestamped data and the selection of an appropriate prediction method. The resulting predictions will be evaluated against observed soil moisture behavior.

The proposal will not assume that reliable watering predictions can be generated without suitable data and validation.

4.5.3 Enhanced Cloud Automation

AWS IoT Rules or AWS Lambda may be investigated as additional cloud components for routing MQTT messages, triggering processing workflows, or supporting event-driven operations. These services will be considered according to their technical suitability and operational cost.

4.5.4 Future System Expansion

The modular architecture will provide a foundation for potential future extensions, such as monitoring multiple plants, introducing additional environmental sensors, or incorporating physical watering control.

Such extensions will remain outside the confirmed core implementation unless they are explicitly selected and included in the finalized project scope. The proposed advanced features will be evaluated according to their feasibility and contribution to the overall GreenPulse system.

CHAPTER 5: IMPLEMENTATION METHODOLOGY

5.1 Development Methodology

The proposed GreenPulse system will be developed using an incremental prototyping methodology. This approach will allow the project team to design, implement, test, and integrate individual components progressively before combining them into a complete IoT-based intelligent plant-care system.

The development process will be divided into six major phases: hardware preparation, embedded firmware development, secure communication configuration, cloud and intelligent backend development, dashboard implementation, and system integration and evaluation.

Each phase will produce identifiable deliverables that can be verified before proceeding to the next stage. This approach will help the team identify technical problems early and reduce the complexity of integrating multiple hardware and software technologies.

5.1.1 Phase 1: Hardware Preparation and Circuit Development

The first phase will focus on identifying and preparing the hardware components required for the GreenPulse prototype. The project team will finalize the component inventory and verify the specifications of the ESP32, soil moisture sensor, DHT22 temperature and humidity sensor, BH1750 light intensity sensor, OLED display, RGB LED, and any additional components included in the original design. A circuit diagram will be prepared to define the electrical connections between the ESP32 and its peripherals. The design will include GPIO assignments, power distribution, common grounding, and any required supporting components.

The hardware will initially be assembled on a breadboard to simplify component replacement and circuit modifications during development. Each sensor will be tested individually before the complete circuit is assembled. The expected outcome of this phase will be a functional hardware prototype capable of collecting environmental measurements and controlling the local output devices.

5.1.2 Phase 2: ESP32 Firmware Development

The second phase will involve developing the embedded firmware responsible for controlling the GreenPulse monitoring device. The firmware will initialize the connected sensors, collect environmental measurements, validate the readings, and update the OLED display.

A priority-based decision mechanism will be developed to control the RGB LED according to predefined environmental conditions. The firmware will also implement Wi-Fi connectivity and prepare sensor measurements as structured JSON messages.

Individual firmware functions will be tested before being integrated into the main application. The OLED display will be restricted to live sensor readings and MQTT connection status. AI-generated care messages and MQTT payload contents will not be displayed on the device screen.

5.1.3 Phase 3: Secure MQTT Communication

The third phase will establish secure communication between the ESP32 and AWS IoT Core. The project team will configure the AWS IoT Core endpoint, register the device, prepare the required authentication credentials, and define appropriate access policies.

The ESP32 firmware will be configured to establish an MQTT connection using TLS. MQTT topics will be created for sensor telemetry, device status, AI-generated care messages, weather summaries, notification summaries, and any selected command functionality.

The team will verify that the ESP32 can publish sensor measurements and receive messages through the designated MQTT topics. Connection recovery mechanisms will also be developed to handle temporary Wi-Fi and MQTT failures.

5.1.4 Phase 4: Cloud Backend and LLM Development

The fourth phase will focus on developing the cloud-hosted intelligent processing backend. A Python-based application will be developed to subscribe to sensor telemetry messages through AWS IoT Core. The backend will validate incoming JSON messages and store timestamped environmental measurements in the proposed Amazon DynamoDB database. The application will also integrate a real weather API and an authorized email or notification account to retrieve relevant external information.

An LLM-based processing framework will be incorporated to generate plant-condition-inspired literary quotes, contextual care tips, weather summaries, and notification summaries. The backend will validate generated outputs and publish the results through designated MQTT topics. Authentication credentials and sensitive configuration information will be managed securely.

5.1.5 Phase 5: Node-RED Dashboard Development

The fifth phase will involve developing the Node-RED visualization dashboard. The dashboard will connect securely to the MQTT broker and subscribe to the relevant telemetry and AI response topics. It will display live environmental measurements, MQTT connection status, topic names, timestamps, LLM input and output payloads, and AI-generated plant-care information.

The proposed dashboard will contain dedicated sections for sensor monitoring, device status, care tips, literary quotes, weather summaries, and notification summaries. The dashboard will be tested using actual MQTT messages generated by the integrated system.

5.1.6 Phase 6: System Integration and Evaluation

The final development phase will combine the hardware, firmware, MQTT broker, cloud backend, external services, LLM processing component, and Node-RED dashboard. The project team will verify that sensor measurements can travel from the ESP32 to the cloud backend and that AI-generated responses can be published through MQTT. The team will also verify that the dashboard displays the required information and that the ESP32 local display follows the assignment restrictions.

Functional tests will be performed to identify communication failures, invalid sensor readings, backend processing errors, and dashboard synchronization problems. The results of these tests will be documented during implementation and used to evaluate the completed prototype.

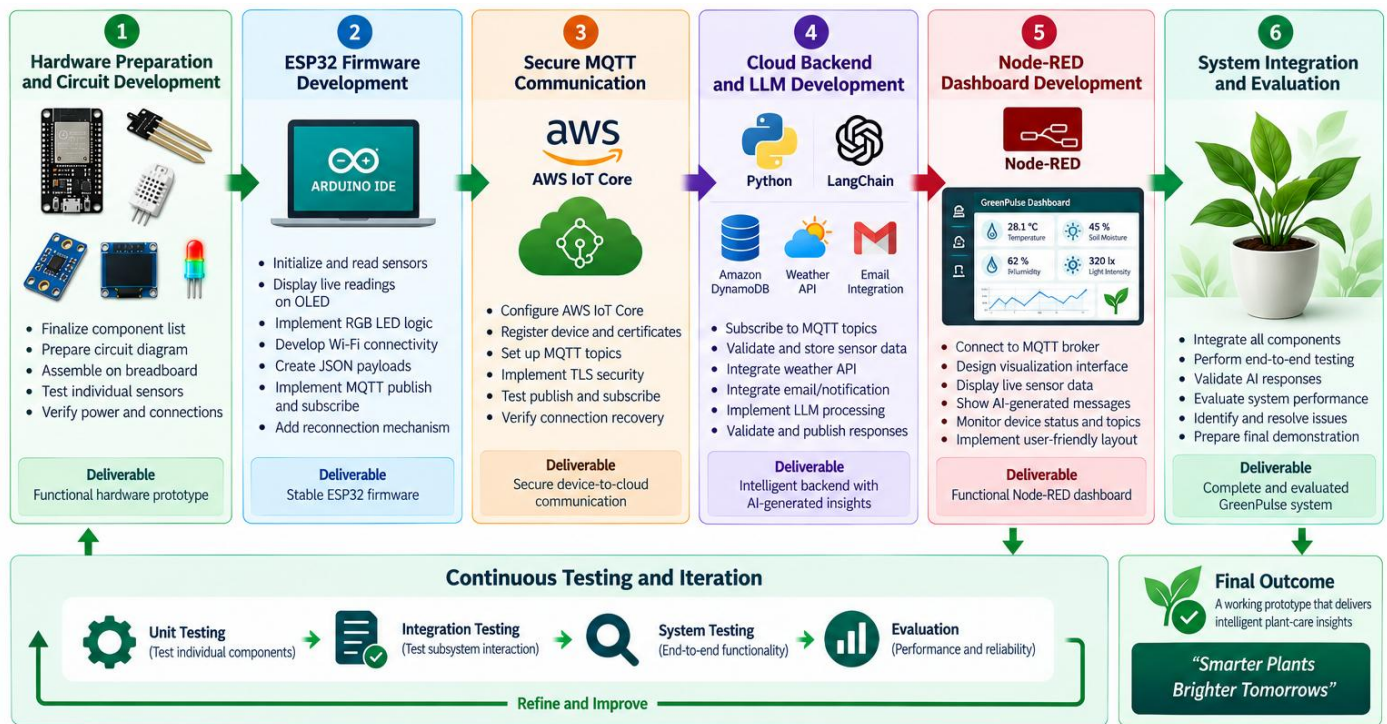


Figure 9 - Proposed Development Methodology of GreenPulse

5.2 System Integration Strategy

System integration will be performed incrementally to reduce the complexity of combining multiple hardware and software components.

The first integration stage will connect the sensors, OLED display, and RGB LED to the ESP32.

The second stage will connect the ESP32 to AWS IoT Core using secure MQTT communication.

The third stage will connect the MQTT broker to the cloud backend and establish timestamped sensor-data storage.

The fourth stage will integrate the LLM service, real weather API, and authorized email account.

The fifth stage will connect the Node-RED dashboard to the MQTT communication infrastructure.

The final stage will verify the complete end-to-end operation of the system.

This staged approach will allow individual subsystems to be tested before the complete architecture is evaluated.

5.3 Testing and Evaluation Plan

A structured testing strategy will be used to evaluate the functionality of the proposed GreenPulse system. The testing process will include hardware testing, firmware testing, communication testing, cloud backend testing, intelligent processing evaluation, dashboard testing, and end-to-end integration testing.

Table 8 - Proposed System Testing and Evaluation Plan

Test ID	Component	Test Procedure	Expected Result
T01	Soil moisture sensor	Collect readings under selected dry and wet reference conditions.	Sensor values change consistently with the tested conditions.
T02	DHT22	Read temperature and humidity measurements.	Valid temperature and humidity values are obtained.
T03	BH1750	Measure light intensity under different lighting conditions.	Light intensity readings respond to changes in illumination.
T04	OLED display	Update sensor values and change MQTT connection state.	Live readings and MQTT status are displayed correctly.
T05	RGB LED	Apply predefined normal, warning, and critical conditions.	The correct priority color is displayed.
T06	Secure MQTT	Connect the ESP32 to AWS IoT Core using TLS.	The device establishes an authenticated connection.
T07	MQTT publishing	Publish sensor telemetry messages.	The broker receives valid JSON messages on the correct topic.
T08	MQTT subscription	Publish a message to a subscribed response topic.	The ESP32 receives the message without displaying AI text on the OLED.
T09	Cloud backend	Send valid and invalid sensor payloads.	Valid data is processed and invalid data is handled appropriately.
T10	Data storage	Publish sensor measurements and inspect the database.	Timestamped records are stored and can be retrieved.
T11	LLM processing	Submit structured sensor measurements.	The backend generates a correctly formatted care tip and literary quote.
T12	Weather API	Request current weather information.	The backend retrieves actual weather data or handles service errors.
T13	Email integration	Retrieve relevant messages from the authorized account.	Relevant information is obtained and summarized without exposing credentials.
T14	Node-RED dashboard	Publish sensor and AI messages.	Required measurements, payloads, topics, timestamps, and status are displayed.
T15	End-to-end system	Execute the complete monitoring and processing workflow.	Data travels from the device to the cloud and generated responses become available through MQTT and the dashboard.

The testing plan will be updated to include any additional sensors or advanced features selected during hardware finalization.

5.3.1 Sensor Evaluation

Sensor evaluation will focus on verifying that each sensor provides valid and responsive measurements. The soil moisture sensor will be calibrated using suitable reference conditions. The DHT22 and BH1750 will be tested under controlled changes in their respective environmental conditions. The project team will record the observations and identify invalid or inconsistent measurements.

5.3.2 Communication Evaluation

MQTT communication will be evaluated by verifying successful connection establishment, message publication, topic subscription, and reconnection behavior. The team will confirm that the selected authentication and TLS configuration allows authorized communication. The system will also be tested under temporary network interruption to observe its recovery behavior.

5.3.3 Intelligent Processing Evaluation

The LLM component will be evaluated using structured sensor inputs and retrieved external information. The evaluation will examine whether the generated responses follow the required JSON structure, use the supplied information, and provide relevant plant-care content. The team will also test missing measurements, unavailable weather data, and unsuccessful email retrieval.

5.3.4 Dashboard Evaluation

The Node-RED dashboard will be evaluated against the visualization requirements. The team will verify the presentation of live sensor readings, MQTT connection status, topic names, timestamps, LLM input payloads, LLM output payloads, and generated summaries. The dashboard will also be checked for readability and appropriate handling of unavailable data.

5.4 Proposed Project Timeline

The GreenPulse project will follow a phased development schedule. The timeline below presents a proposed four-week implementation plan. The actual dates and duration will be adjusted according to the assignment schedule and the group's available development time.

Table 9 - Proposed Project Implementation Timeline

Activity	Week 1	Week 2	Week 3	Week 4
Requirements and hardware finalization	✓			
Circuit assembly and sensor testing	✓	✓		
ESP32 firmware development	✓	✓		
Secure MQTT configuration		✓		
Cloud backend and database		✓	✓	

LLM integration			✓	
Weather and email integration			✓	
Node-RED dashboard			✓	✓
System integration				✓
Testing and debugging		✓	✓	✓
Documentation and demonstration preparation	✓	✓	✓	✓
Commercial product video				✓

The implementation schedule will prioritize the mandatory hardware, secure communication, cloud processing, and dashboard requirements before optional extensions.

5.5 Group Member Responsibilities

The project will require coordination between hardware development, embedded programming, cloud integration, intelligent processing, and dashboard implementation. Responsibilities will be assigned according to the final number of group members and their technical capabilities.

Table 10 - Proposed Group Member Responsibilities

Member	Primary Responsibilities
Member 1	Hardware component selection, circuit design, sensor integration, and hardware testing.
Member 2	ESP32 firmware, OLED display, RGB LED logic, Wi-Fi connectivity, and secure MQTT communication.
Member 3	Cloud backend, database integration, LLM processing, weather API, and email integration.
Member 4	Node-RED dashboard, MQTT visualization, system integration support, testing documentation, and demonstration preparation.

All members will contribute to overall system integration, testing, documentation, and the final demonstration. Individual contributions will be documented to support the assignment's individual assessment requirements.

5.6 Budget and Feasibility

The proposed GreenPulse system will require hardware components, software development tools, cloud services, and external API integrations. A bill of materials will be prepared after confirming the exact component models and quantities.

Table 11 - Proposed Bill of Materials and Budget Structure

Component or Service	Quantity	Estimated Cost (LKR)
ESP32 development board	1	To be confirmed
Soil moisture sensor	1	To be confirmed
DHT22 sensor	1	To be confirmed
BH1750 sensor	1	To be confirmed
OLED display	1	To be confirmed
RGB LED and required resistors	1 set	To be confirmed
Breadboard, jumper wires, and connectors	1 set	To be confirmed
Regulated power supply	1	To be confirmed
Additional sensors and supporting components	As required	To be confirmed
AWS cloud services	As required	Usage-dependent
LLM service	As required	Usage-dependent
Weather API	As required	Plan-dependent
Total		To be calculated

The hardware budget will be calculated using verified component prices from selected suppliers. Cloud and API expenses will be estimated according to the selected service plans, anticipated usage, and available free-tier allowances.

5.6.1 Technical Feasibility

The proposed system will use established hardware and software technologies that support the required sensing, communication, and processing functions. The ESP32 will provide the embedded processing and wireless connectivity capabilities, while AWS IoT Core will provide the proposed MQTT communication infrastructure.

The Python backend, selected LLM service, and Node-RED dashboard will support the intelligent processing and visualization requirements. Technical feasibility will depend on successful integration of the selected components and services.

5.6.2 Economic Feasibility

The project will aim to control costs by using a compact prototype, commercially available sensors, and suitable cloud service configurations.

The team will monitor the anticipated cost of cloud computing, database operations, API requests, and LLM usage. Paid services will be selected only after considering the project's functional requirements and available budget.

5.7 Risk Management and Mitigation

The development of GreenPulse may involve hardware, communication, cloud integration, and intelligent processing challenges. Potential risks will be identified before implementation, and appropriate mitigation strategies will be incorporated into the development plan.

Table 12 - Proposed Project Risks and Mitigation Strategies

Risk	Potential Impact	Proposed Mitigation
Incorrect sensor readings	Unreliable environmental information	Calibrate sensors and validate readings before processing.
Incorrect circuit connections	Component malfunction or damage	Verify wiring, voltage levels, and power distribution before powering the circuit.
Wi-Fi disconnection	Interrupted communication	Implement connection monitoring and reconnection procedures.
MQTT authentication failure	Device cannot connect to broker	Verify certificates, policies, endpoint configuration, and client settings.
Cloud service interruption	Delayed data processing	Implement error handling and suitable retry mechanisms.
LLM output errors	Invalid or unsupported care information	Use structured prompts, validate output, and avoid unsupported claims.
Weather API failure	Missing external weather information	Detect failures and prevent unavailable data from being presented as current information.
Email authentication failure	Notification retrieval unavailable	Use supported authentication and handle token or connection errors.
Cloud or API cost increase	Budget overrun	Monitor usage and configure appropriate service limits.
Integration delays	Reduced time for evaluation	Develop incrementally and prioritize mandatory features.

Risk management will be reviewed throughout the development process. The project team will prioritize reliable sensor acquisition, secure communication, correct data processing, and compliance with the assignment requirements before implementing optional features. The proposed methodology will provide a structured foundation for developing, integrating, and evaluating the GreenPulse prototype.

CHAPTER 6: EXPECTED OUTCOMES AND CONCLUSION

6.1 Expected Outcomes

The proposed GreenPulse project is expected to demonstrate the integration of Internet of Things (IoT) technology, secure cloud communication, environmental monitoring, and Large Language Model (LLM)-based information processing within a unified plant-care system.

The project will aim to produce a functional prototype capable of collecting environmental measurements, securely transmitting sensor data, generating contextual plant-care information, and presenting the results through a centralized visualization dashboard. The anticipated outcomes of the project are described below.

6.1.1 Functional IoT Monitoring Device

The project is expected to produce an ESP32-based monitoring device capable of collecting environmental measurements from the connected sensors. The proposed system will measure soil moisture, ambient temperature, relative humidity, and light intensity using the selected sensor modules. Measurements from any additional sensors included in the finalized GreenPulse design will also be incorporated.

The ESP32 will process the collected information and transmit the sensor measurements to the selected MQTT broker. This outcome will demonstrate the proposed integration of multiple environmental sensors with an embedded IoT controller.

6.1.2 Local Monitoring and Priority Indication

The prototype is expected to provide immediate environmental information through an OLED display. The display will present live sensor measurements and MQTT connection status, allowing users to observe the current monitoring conditions and device connectivity. An RGB LED will indicate predefined plant-care priority levels using normal, warning, and critical states.

The priority indication mechanism will be evaluated using selected sensor conditions and threshold values. The OLED display will remain restricted to live sensor readings and MQTT connection status. AI-generated care messages and MQTT payload contents will not be displayed on the device screen.

6.1.3 Secure Bidirectional MQTT Communication

The proposed system is expected to establish secure communication between the ESP32, MQTT broker, and cloud backend. The ESP32 will publish environmental measurements through defined MQTT topics using structured JSON payloads.

The cloud backend will subscribe to the sensor telemetry topics and publish generated responses to separate topics. Transport Layer Security (TLS) and suitable authentication mechanisms will be implemented to protect communication. The expected outcome will be a bidirectional communication architecture that supports device-to-cloud telemetry and cloud-to-device messaging.

6.1.4 Cloud-Based Data Collection and Storage

The project is expected to implement a cloud-hosted backend capable of receiving, validating, and storing environmental sensor measurements. Timestamped records will be maintained using the selected database service.

The stored information will support historical data retrieval, system monitoring, troubleshooting, and potential future analytical functions. The backend will also provide the processing infrastructure required for integrating external information sources and LLM-based services.

6.1.5 LLM-Generated Plant-Care Information

A major expected outcome of GreenPulse is the generation of contextual plant-care information using an LLM-based backend. The system will use structured environmental sensor measurements to generate concise care tips and plant-condition-inspired literary quotes. The LLM will be instructed to use the available information and avoid inventing unavailable measurements or unsupported observations.

The generated responses will be validated and published through designated MQTT topics. This functionality is expected to demonstrate how LLM technology can extend conventional IoT monitoring by converting numerical measurements into understandable natural-language information.

6.1.6 Real Weather and Notification Summarization

The proposed backend is expected to integrate a real weather API and an authorized email or notification account. Weather information will be retrieved and processed to generate summaries relevant to plant-care activities. The system will also retrieve relevant messages from the authorized account and produce concise notification summaries.

The generated information will be published through MQTT and made available through the Node-RED dashboard. This outcome will demonstrate the integration of external real-world information with environmental sensor monitoring and LLM-based processing.

6.1.7 Centralized Node-RED Dashboard

The project is expected to produce a Node-RED dashboard capable of displaying live environmental measurements and information exchanged through MQTT. The dashboard will present sensor values, MQTT connection status, topic names, timestamps, LLM input payloads, LLM output payloads, and generated plant-care information.

Separate interface sections will be provided for care tips, literary quotes, weather summaries, and notification summaries. The dashboard will support system monitoring and provide a centralized interface for demonstrating the operation of GreenPulse.

6.1.8 Integrated End-to-End Prototype

The final expected outcome will be an integrated prototype that demonstrates the complete GreenPulse processing workflow. The ESP32 will collect environmental measurements and transmit them securely through MQTT. The cloud backend will receive and store the information, retrieve relevant external data, and coordinate LLM-based processing.

The generated responses will be published through MQTT and displayed through the Node-RED dashboard. The completed prototype will be evaluated against the functional and integration requirements established in the project proposal.

6.2 Project Deliverables

The GreenPulse project will produce a collection of hardware, software, documentation, and demonstration deliverables.

These deliverables will provide evidence of the proposed system's design, implementation, and integration.

Table 13 - Proposed GreenPulse Project Deliverables

No.	Deliverable	Description
D01	IoT Hardware Prototype	ESP32-based monitoring device integrating the selected environmental sensors, OLED display, and RGB LED.
D02	ESP32 Firmware	Source code for sensor acquisition, local visualization, priority indication, Wi-Fi connectivity, and secure MQTT communication.
D03	MQTT Configuration	Topic definitions, authentication configuration, access policies, and communication setup instructions.
D04	Cloud Backend	Cloud-hosted application for MQTT message processing, data validation, external API integration, and LLM coordination.
D05	Data Storage	Configured database for storing timestamped environmental sensor measurements.
D06	LLM Processing Component	Implementation for generating plant-care tips, literary quotes, weather summaries, and notification summaries.
D07	Node-RED Dashboard	Dashboard interface and exportable Node-RED flow configuration.
D08	Technical Documentation	System architecture, hardware connections, MQTT payload formats, configuration details, and deployment instructions.
D09	Live Demonstration	End-to-end demonstration of sensor acquisition, secure communication, cloud processing, and dashboard visualization.
D10	Commercial Product Video	Promotional video presenting the GreenPulse concept, major features, intended users, and potential practical applications.
D11	Final Project Report	Report documenting the implemented system, architecture, challenges, testing evidence, and results according to the assignment requirements.

The deliverables will be prepared according to the assignment requirements and the actual functionality achieved during development. The final implementation report will be produced separately from this project proposal and will document the completed work and verified results.

6.3 Expected Project Contribution

GreenPulse is expected to demonstrate a practical approach to combining embedded IoT monitoring with cloud-hosted intelligent information processing. The proposed system will integrate multiple environmental sensors, secure MQTT communication, cloud data storage, external information retrieval, and LLM-generated responses within a single architecture.

The project will also demonstrate the separation of responsibilities between an embedded monitoring device and a cloud-based intelligent backend. The ESP32 will perform sensor acquisition and local status indication, while the cloud backend will handle data storage, external information integration, and language generation.

This separation is expected to support modular development and simplify the integration of additional processing capabilities. The Node-RED dashboard will provide visibility into the complete communication and processing workflow, including the information submitted to the LLM and the responses generated by the backend.

From an educational perspective, the project is expected to provide practical experience in embedded systems, IoT communication protocols, cloud computing, API integration, LLM application development, and system integration. The proposed system will serve as a prototype demonstrating how these technologies can be combined to provide contextual information for plant maintenance.

6.4 Future Development Opportunities

The GreenPulse architecture will be designed to accommodate potential extensions beyond the initial prototype. One possible enhancement will involve monitoring multiple plants using additional ESP32 devices and identifying each device through a unique identifier. Another potential extension will involve collecting historical environmental measurements and developing a data-driven watering prediction mechanism.

The system may also be extended to support automatic watering through suitable actuators and additional safety mechanisms. Such functionality will require further hardware development, control logic, and testing. Additional environmental sensors may be integrated to expand the information available to the intelligent processing backend.

Further development may also investigate improved plant-specific care recommendations, more advanced cloud automation, and additional user interaction mechanisms. These enhancements will be considered future development opportunities unless they are explicitly included in the finalized implementation scope.

6.5 Conclusion

This project proposal has presented the proposed design and development approach for GreenPulse, a smart IoT plant-care system integrating environmental monitoring, secure MQTT communication, cloud computing, and Large Language Model technology.

The proposed system will use an ESP32 microcontroller and environmental sensors to collect measurements related to soil moisture, temperature, humidity, and light intensity. An OLED display will provide live sensor readings and MQTT connection status, while an RGB LED will indicate predefined plant-care priority levels.

Sensor measurements will be transmitted securely through MQTT to a cloud-hosted backend, where the information will be validated and stored with timestamps.

The backend will integrate an LLM-based processing framework, a real weather API, and an authorized email or notification account to generate contextual plant-care messages, literary quotes, weather summaries, and notification summaries.

A Node-RED dashboard will provide centralized visualization of environmental measurements, MQTT communication information, and AI-generated responses. The proposed implementation methodology will support incremental development, subsystem integration, and structured testing.

Through these components, GreenPulse aims to demonstrate an integrated approach to transforming environmental sensor measurements into accessible and context-aware plant-care information. The successful implementation and evaluation of the proposed prototype will be determined through the planned testing and demonstration activities. The project is expected to provide a foundation for further development of intelligent IoT-based plant-monitoring applications.